MACHINE LEARNING BASED SERVICE LIFE AND WHOLE-LIFE COST PREDICTION FOR PRE-CONSTRUCTION DECISION SUPPORT

INDIVIDUAL THESIS

Shamindi E. E.

(IT22547460)

Individual component: Component 4 – Cost and Lifecycle Consultant (Service Life Prediction and Whole-Life Costing)

Group ID: R26-IT-154

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) in Information Technology

Specialising in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

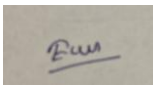
DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Shamindi E. E.

Student ID: IT22547460

Signature:  Date: 10/11/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	**Signature**	**Date**
Supervisor: Mr. Ravi Supunya		
Co-supervisor: Mrs. Suriya Kumari		

ABSTRACT

Pre-construction cost assessment in Sri Lanka stops at capital expenditure, so a building that is cheaper to build but needs major structural intervention twice as often is recorded as cheaper. Whole-life costing frameworks such as ISO 15686-5 would answer the question, but they take service life as an input, and for a building that has not been built nobody can supply it. Practitioners fall back on a flat five percent depreciation rule that says the same about every building.

This thesis presents the Cost and Lifecycle Consultant, Component 4 of the ConstructAI platform, which converts the other three analyses into a decision. It assembles a thirteen-parameter vector describing a building's fabric, environment and maintenance regime, predicts structural service life with a gradient-boosted regressor trained on a physics-informed dataset of 1 000 records, prices the bill of quantities at automatically collected rates, and computes a discounted fifty-year cost trajectory with replacement events at predicted end of life. Specification alternatives are reported as a break-even year.

On a held-out partition the model reaches a coefficient of determination of 0.9847 and a root mean squared error of 0.433 years, 91.5 percent of predictions within a year of target. Three findings run against the author's design choices. Only five of the thirteen parameters carry information, because the degradation rules written for this work use only five; workmanship and material quality alone account for 82.5 percent of model gain. The deployed configuration is outperformed by a shallower tree depth and uses four times more trees than needed. And whole-life costing does reverse the specification chosen on capital cost alone, but by only 1.1 percent over fifty years with a break-even at year 14, because discounting is unforgiving to late benefits. The training data is synthetic, so these figures characterise the formulation rather than real buildings.

Keywords: service life prediction, whole-life costing, XGBoost, construction cost estimation, value engineering, decision support

ACKNOWLEDGEMENTS

I thank my supervisor for guidance throughout this project and in particular for two interventions that changed it. The first was the five percent depreciation heuristic, which gave the degradation function somewhere to start. The second was the insistence that I evaluate on a held-out partition instead of reporting the fit on the full dataset, which lowered my headline number and made it defensible. I thank my co-supervisor for detailed comments on the methodology and results chapters.

I am grateful to the practising engineers and quantity surveyors who gave time to a student project during the requirements stage, and who were candid about how little of a building's real cost appears in a bill of quantities. The break-even framing that the interface now uses came out of one of those conversations rather than out of the literature.

I thank the maintainers of XGBoost, scikit-learn, Pandas and NumPy, and the UCI Machine Learning Repository for the concrete strength data that anchored the quality terms in the degradation function. Work of this kind at undergraduate level is only possible because other people chose to make these things free.

Finally I thank my three project teammates. Building four components that genuinely interoperate is much harder than building four that merely coexist, and as the last component in the chain I depended on the other three more than any of them depended on me. The interface-first discipline we agreed in the second sprint is the reason it worked.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION ..... **i**

ABSTRACT ..... **ii**

ACKNOWLEDGEMENTS ..... **iii**

LIST OF FIGURES ..... **vii**

LIST OF TABLES ..... **viii**

LIST OF ABBREVIATIONS ..... **ix**

1 INTRODUCTION **1**

1.1 Overall project context 1

1.2 Individual component context 2

1.3 Problem statement 3

1.4 Research gap 3

1.5 Aim and objectives 4

1.6 Research questions or hypotheses 5

1.7 Scope limitations and assumptions 6

1.8 Individual contribution 6

1.9 Thesis structure 7

2 LITERATURE REVIEW AND RELATED WORK **8**

2.1 Review method and scope 8

2.2 Domain and theoretical foundations 8

2.2.1 Gradient boosted trees 8

2.2.2 How buildings deteriorate 8

2.2.3 Whole-life costing 9

2.2.4 The five percent heuristic 9

2.3 Related research 10

2.3.1 Machine learning for construction cost 10

2.3.2 Prediction beyond handover 10

2.3.3 Interpretability of tabular models 10

2.4 Existing systems products and techniques 11

2.5 Gap synthesis and conceptual framework 11

3 RESEARCH METHODOLOGY **13**

3.1 Research design 13

3.2 Data participants or study materials 14

3.3 Experimental or study procedure 14

3.3.1 The degradation function 14

3.3.2 Dataset generation 15

3.3.3 Model selection, training and tuning 16

3.3.4 Lifecycle cost computation 16

3.3.5 Value engineering and advisory logic 16

3.3.6 Sensitivity analysis 17

3.4 Measures and evaluation metrics 17

3.5 Data analysis 18

3.6 Ethics privacy security and responsible AI 18
3.7 Reproducibility and research artefacts 19
4 PRODUCT AND SOLUTION DESIGN 20
4.1 Requirements 20
4.2 Component architecture within the complete solution 21
4.3 Design alternatives and decisions 21
4.3.1 A learned model rather than the heuristic 21
4.3.2 A learned model rather than a physics model 21
4.3.3 XGBoost rather than a simpler regressor 22
4.3.4 Additive penalties rather than multiplicative interaction 22
4.3.5 Normalised rather than absolute cost reporting 22
4.4 Detailed design 22
4.4.1 Feature assembly 22
4.4.2 Service life prediction 23
4.4.3 Capital cost 23
4.4.4 Lifecycle discounting 24
4.4.5 Value engineering 24
4.4.6 Advisory layer 24
4.4.7 The specification block contract 24
4.5 Standards and professional practice 25
5 IMPLEMENTATION INTEGRATION AND TESTING 26
5.1 Development environment and technology stack 26
5.2 Component implementation 26
5.2.1 Dataset generation 26
5.2.2 Feature assembly and validation 27
5.2.3 Model training and serving 27
5.2.4 Lifecycle computation 27
5.2.5 Advisory layer 27
5.2.6 Explanation output 28
5.3 Integration with the other components 28
5.4 Verification and software testing 29
5.5 Deployment and operation 30
6 RESULTS AND VALIDATION 31
6.1 Research results 31
6.1.1 Model comparison 31
6.1.2 Hyperparameter sensitivity 33
6.1.3 Feature importance 35
6.1.4 Eight features do nothing, and that is my fault 36
6.1.5 Comparison against the five percent heuristic 37
6.1.6 Learning curve 38
6.2 Product and technical results 39
6.3 Whole-life costing and value engineering 40
6.4 User or stakeholder validation 42
6.5 Integration results 42
6.6 Summary of findings 43

7 DISCUSSION **44**
7.1 Interpretation of findings 44
7.2 Comparison with literature baselines and existing solutions 44
7.3 Research contribution 45
7.4 Product contribution 46
7.5 Limitations and threats to validity 46
7.5.1 The training data is synthetic, and I generated it 46
7.5.2 Five of thirteen features carry the model 46
7.5.3 The dominant inputs are self-assessed 47
7.5.4 Additive penalties and bounded quality effect 47
7.5.5 Cost parameters are assumptions 47
7.5.6 Price collection is brittle 47
7.6 Implications 47
8 COMMERCIALISATION AND SUSTAINABILITY **49**
8.1 Value proposition and target users 49
8.2 Market and competitive position 49
8.3 Business or sustainability model 50
8.4 Scalability intellectual property and compliance 50
8.5 Contribution to the Sustainable Development Goals 51
9 CONCLUSION AND FUTURE DIRECTIONS **52**
9.1 Conclusion 52
9.2 Achievement of objectives 52
9.3 Individual research and product contributions 53
9.4 Future directions 53
REFERENCES **55**
GLOSSARY **57**
APPENDICES **58**
Appendix A Extended results and research instruments 58
A.1 Feature distributions used in dataset generation 58
A.2 Generated target distribution 58
A.3 Full partial dependence results 59
A.4 Degradation rules as implemented 59
A.5 Lifecycle cost parameters 60
A.6 User acceptance protocol 60
Appendix B Academic integrity and AI use disclosure 60
B.1 Statement of original contribution 61

LIST OF FIGURES

Figure 1.1: Architecture of the ConstructAI platform, with Component 4 as the decision hub. 2
Figure 2.1: Processing pipeline of the Cost and Lifecycle Consultant. 12
Figure 6.1: Held-out R² across the six evaluated regressors. 31
Figure 6.2: Predicted against actual service life, held-out test partition. 32
Figure 6.3: Distribution of prediction residuals, held-out test partition. 32
Figure 6.4: Held-out R² across the tree depth and ensemble size grid. 33
Figure 6.5: Gain-based importance of the thirteen input features. 35
Figure 6.6: Share of total model gain by source component. 35
Figure 6.7: Predicted service life against the flat five percent depreciation rule. 37
Figure 6.8: Held-out R² against the number of training records used. 38
Figure 6.9: Fifty-year discounted cost trajectory, as-drawn against upgraded specification. 40

LIST OF TABLES

Table 2.1: Comparison of existing approaches to pre-construction cost assessment 10
Table 3.1: Study materials used by Component 4 14
Table 3.2: Adjustments in the degradation rules and their justification 14
Table 3.3: Evaluation measures and the question each one answers 17
Table 4.1: Functional requirements for Component 4 20
Table 4.2: Non-functional requirements for Component 4 20
Table 4.3: The thirteen input features, their domains and their sources 22
Table 4.4: Specification block written by Component 4 24
Table 5.1: Technology stack for Component 4 26
Table 5.2: Verification performed at each test level 29
Table 6.1: Regression performance on the held-out test partition (n = 200) 31
Table 6.2: Held-out R^2 across the hyperparameter grid 33
Table 6.3: Gain-based feature importance, all thirteen features 35
Table 6.4: Model predictions against the flat twenty-year heuristic 37
Table 6.5: Learning curve of the service life regressor 38
Table 6.6: Non-functional requirements and how each was verified 39
Table 6.7: Value engineering comparison on a coastal, acidic-soil site 40
Table 6.8: User acceptance tasks relating to the lifecycle output 42
Table 9.1: Achievement of the seven objectives stated in Section 1.5 52
Table A.1: Sampling distributions for the thirteen features 58
Table A.2: Distribution of the generated service life target 58
Table A.3: Configurable parameters of the lifecycle computation 60

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AGPL	Affero General Public License
API	Application Programming Interface
BOQ	Bill of Quantities
CAPEX	Capital Expenditure
CPU	Central Processing Unit
CV	Cross-Validation
GBM	Gradient Boosting Machine
JSON	JavaScript Object Notation
LCC	Life-Cycle Cost
LKR	Sri Lankan Rupee
MAE	Mean Absolute Error
MPa	Megapascal
MSL	Mean Sea Level
NFR	Non-Functional Requirement
NPV	Net Present Value
OPEX	Operational Expenditure
RC	Reinforced Concrete
RMSE	Root Mean Squared Error
SDG	Sustainable Development Goal
SHAP	SHapley Additive exPlanations
SLIIT	Sri Lanka Institute of Information Technology
SVR	Support Vector Regression
UAT	User Acceptance Testing
UDA	Urban Development Authority (Sri Lanka)

Abbreviation	Meaning
XGBoost	Extreme Gradient Boosting

1 INTRODUCTION

1.1 Overall project context

ConstructAI is a group research project built around a complaint we heard from almost every practitioner we spoke to during the proposal stage: the four analyses that happen before a building is built are done by four different people who never see each other's work. A surveyor assesses the land. An architect draws the building. A lawyer clears the title and the zoning. A quantity surveyor measures the drawings and prices them. Each produces a document and hands it on, and documents do not recompute when something upstream changes.

What the industry pays for that is overruns. Across a large international sample of projects, roughly nine in ten exceeded their approved budget with mean overruns near 28 percent, a pattern too consistent across decades and countries to be dismissed as estimating noise [1]. Smaller residential projects, which make up most construction activity in Sri Lanka, are not exempt, and early-stage manual takeoff under time pressure is commonly reported to carry errors in the 10 to 15 percent range [2].

Our group built a single platform that performs all four analyses over one shared project record, so that the output of each constrains the others. Component 1 turns a land coordinate into an environmental profile. Component 2 turns a floor plan into measured quantities and structural parameters. Component 3 checks the resulting design against the Urban Development Authority regulations. Component 4, which is mine, takes everything the first three produce and answers the question the client actually asked: what will this building cost me, not to build, but to own. Figure 1.1 shows the arrangement.

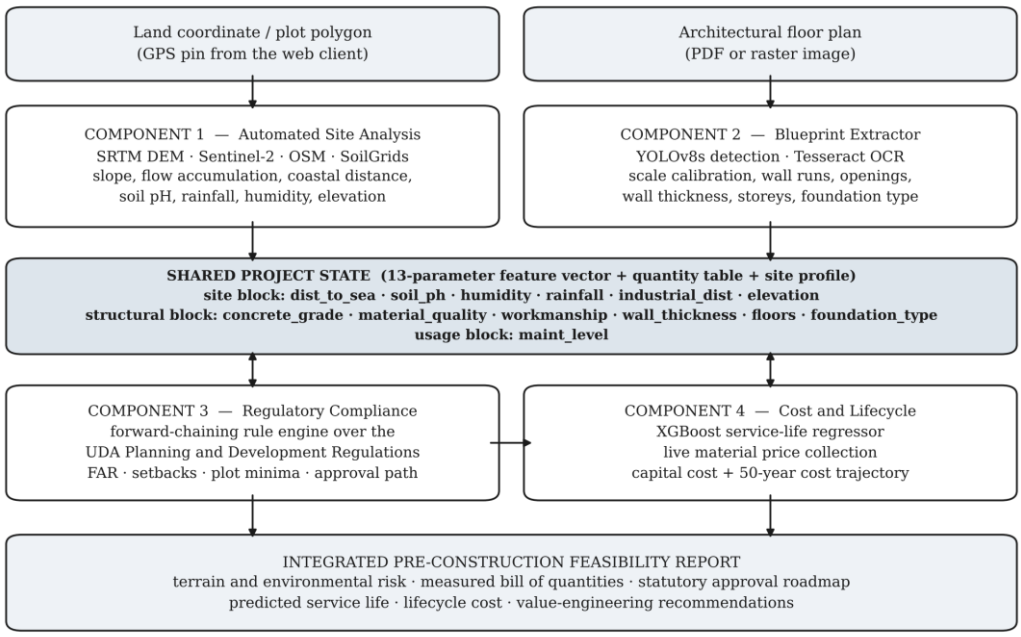


Figure 1.1: Architecture of the ConstructAI platform, with Component 4 as the decision hub.

1.2 Individual component context

My component is the Cost and Lifecycle Consultant. It does three things. It assembles the thirteen-element parameter vector that describes a specific building on a specific site. It predicts how long that building will last, using a gradient-boosted regression model trained on a physics-informed dataset. And it converts that prediction into money, by pricing the bill of quantities at current material rates and discounting the maintenance and replacement events that follow over a fifty-year horizon.

That third step matters because it is the one the industry skips. A conventional bill of quantities computes capital expenditure and stops. Under that convention a building that is four percent cheaper to build and needs its roof replaced twice as often is recorded as the cheaper building, and the client finds out otherwise fifteen years later. ISO 15686-5 has specified the framework for whole-life costing of constructed assets for years [3], and the framework is not the obstacle. The obstacle is that the framework assumes the analyst can supply the service life as an input, and nobody can.

That assumption is what this component attacks. Rather than treating service life as a number the analyst looks up in a table, I treat it as a quantity to be predicted from the specific fabric of the building and the specific aggressiveness of its environment. The starting point was an industry rule of thumb our supervisor gave us, that a building depreciates at roughly five percent a year over a twenty-year baseline. That rule is useful precisely because it is simple, and useless for the same reason: it says the same thing about a well-built house inland and a poorly built one three hundred metres from the sea. Section 6.1.5 shows how far apart those two actually are.

1.3 Problem statement

Pre-construction cost assessment in Sri Lanka stops at capital expenditure, so clients commit to designs and specifications without any estimate of what the building will cost to maintain, repair and eventually replace. The whole-life costing frameworks that would answer the question require a service life figure as an input, and no method exists for deriving that figure from the properties of a specific building on a specific site at the point where the decision is actually made.

In one sentence: there is no automated method that predicts the service life of a proposed low-rise Sri Lankan building from its design and site parameters, and converts that prediction into a decision-useful whole-life cost.

1.4 Research gap

Three literatures bear on this and none of them closes it.

First is data-driven construction cost estimation, a mature field. Gradient boosting in particular has become the default for tabular cost regression: Ali and Burhan compared XGBoost against extreme learning machines and multivariate adaptive regression splines on building project data and reported an R-squared of 0.952 for the boosted model [6]; Zhang and colleagues built a hybrid early-stage estimator [7]; Singh and colleagues stacked random forests with XGBoost for budgeting [8]; Lin addressed unit price prediction for labour [9]. Every one of these predicts capital cost. None predicts what happens after handover.

Second is durability and service life modelling, which has produced rigorous deterioration models. Tuutti's two-phase corrosion model remains the conceptual basis for most of the field [11], and the fib Model Code for Service Life Design formalises chloride ingress and carbonation as diffusion-controlled processes [12]. These models are written for a structural engineer who already knows the exposure class and wants a cover depth. They are not written to be run from a web form by somebody deciding whether to buy a plot, and they require material parameters nobody measures on a small residential project.

Third is whole-life costing itself, standardised in ISO 15686-5 [3] within the wider service life planning framework of ISO 15686-1 [14]. The standard is a method, not a predictor. It tells you how to discount a maintenance schedule; it explicitly assumes somebody hands you the service life and the schedule.

What is missing is the join. Nobody has taken the tabular machine learning approach that works so well for capital cost and pointed it at service life instead, using features that combine the building's fabric with the aggressiveness of its location, and then fed the result into the discounting framework that already exists. That join is Component 4.

1.5 Aim and objectives

Aim. To design, implement and evaluate a machine learning component that predicts the structural service life of a proposed low-rise Sri Lankan building from thirteen design and site parameters, converts that prediction into a discounted fifty-year cost trajectory, and generates value-engineering recommendations by comparing alternative specifications.

1. **O1.** Formalise the degradation behaviour of low-rise Sri Lankan construction as an explicit, auditable function of fabric, environment and maintenance, grounded in the durability literature and in the industry depreciation heuristic.
2. **O2.** Construct a physics-informed dataset from that function sufficient to train and evaluate a supervised regression model, and document honestly what such a dataset can and cannot support.

3. **O3.** Select, train and tune a regression model for service life prediction, and benchmark it against credible baselines on a properly held-out partition.
4. **O4.** Implement capital cost computation from the quantity table against material prices collected automatically, with explicit staleness handling.
5. **O5.** Implement whole-life cost computation following ISO 15686-5, with replacement events placed at predicted end of life rather than at fixed intervals.
6. **O6.** Implement and evaluate value engineering: re-run the model under alternative specifications and determine whether and when a specification premium pays back.
7. **O7.** Quantify the contribution of each input parameter to the prediction, and report which parameters turn out to carry no information.

O7 was added after the first evaluation round, and it produced the most uncomfortable result in this thesis. I have kept it as a stated objective instead of presenting the finding as though it had been planned.

1.6 Research questions or hypotheses

- **RQ1.** Can structural service life be predicted from thirteen design and site parameters with accuracy sufficient to discriminate between specification alternatives?
- **RQ2.** Which model class performs best on this problem, and is the choice empirically justified rather than conventional?
- **RQ3.** How much does each input parameter contribute to the prediction, and do all thirteen earn their place?
- **RQ4.** Does whole-life costing built on a predicted service life change the specification a client should choose, compared with capital cost alone?
- **RQ5.** How much does the prediction improve on the flat five percent depreciation heuristic the industry currently uses?

RQ4 rests on a hypothesis: on an aggressive site a specification upgrade that looks wasteful on capital cost will be justified on whole-life cost. Section 6.3 tests that and the answer is more qualified than I expected.

1.7 Scope limitations and assumptions

Scope is low-rise residential and light-commercial reinforced concrete and masonry construction in Sri Lanka. It predicts structural service life, meaning the point at which major structural repair becomes necessary, not the point at which the building becomes uninhabitable or is demolished for commercial reasons.

Five assumptions carry the weight of the whole component and I would rather put them at the front than let a reader find them in Chapter 7.

- **The training data is generated, not observed.** No corpus of real Sri Lankan buildings with recorded age, specification, location and surveyed condition exists publicly, and assembling one was beyond a one-year project. Everything in Chapter 6 measures how well the model recovers the function I wrote in Section 3.3.
- The degradation function is additive in its penalties. Real deterioration mechanisms interact; chloride ingress and high moisture compound rather than add. Section 7.5 treats this as a modelling simplification I would remove next.
- Material and workmanship quality are supplied as ordinal indices from 1 to 5 by the user, not measured. They turn out to dominate the prediction, which makes this the most consequential assumption in the component.
- Maintenance is assumed constant over the building's life at the level the user declares. Real maintenance regimes lapse.
- The discount rate and the unit cost parameters in the lifecycle computation are configurable inputs, not findings. The cost figures in Section 6.3 are normalised against capital cost for exactly this reason.

1.8 Individual contribution

My contribution is the whole of the cost and lifecycle engine: the degradation function and the dataset generation script that implements it, the feature assembly layer and its validation rules, the service life model with its training, tuning and benchmarking, the capital cost computation and price collection, the ISO 15686-5 discounting, the value engineering comparison, and the advisory logic that turns a prediction into a sentence a client can act on.

Parameter definitions came from the other components: the six environmental parameters and their threshold justifications from the Component 1 owner, and the six structural parameters from the Component 2 owner. I wrote the function that combines them, which means the defect reported in Section 6.1.4, that only five of the thirteen parameters appear in the function at all, is mine and not theirs.

1.9 Thesis structure

Chapter 2 reviews cost estimation, durability modelling and whole-life costing, and sets out the gap. Chapter 3 describes the research design, the degradation function, the dataset and the evaluation protocol. Chapter 4 presents the component design. Chapter 5 covers implementation, integration and testing. Chapter 6 reports the results. Chapter 7 discusses them and states the limitations. Chapter 8 covers commercialisation and sustainability. Chapter 9 concludes.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

Searches were run on IEEE Xplore, ScienceDirect, the ASCE library and Google Scholar between February and July 2026, combining construction cost estimation, machine learning, gradient boosting, service life prediction, whole-life costing, concrete durability and building deterioration, each optionally paired with Sri Lanka or South Asia. Standards were treated as primary sources and read directly.

I kept peer-reviewed work and standards documents, preferred sources from 2015 onward for the machine learning material and retained older work where it is the canonical reference for a method still in use. I excluded the large body of work on cost estimation for infrastructure megaprojects, because the feature sets and the cost drivers do not transfer to a two-storey house.

2.2 Domain and theoretical foundations

2.2.1 Gradient boosted trees

Gradient boosting fits an additive ensemble of regression trees, each trained on the residuals of the ensemble so far, in the stagewise manner formalised by Friedman [5]. Chen and Guestrin's XGBoost refines that scheme in two ways: it uses a second-order approximation of the loss when deciding how to grow each tree, and it penalises tree complexity explicitly, charging the fit for every additional leaf and for large leaf weights, so the ensemble is pushed towards the smallest set of trees that explains the data [4].

Three properties matter for this problem: the method handles heterogeneous tabular features without scaling or encoding gymnastics, that it captures interactions and thresholds without being told they exist, and that it behaves reasonably on datasets of a few thousand rows. Random forests [15] offer similar convenience with a different variance-reduction mechanism and are the natural first baseline.

2.2.2 How buildings deteriorate

For reinforced concrete in a tropical coastal setting the dominant mechanism is corrosion of the embedded reinforcement. Tuutti's model splits the process into an

initiation period, during which chloride or carbonation fronts penetrate the cover concrete while the steel stays passive, and a propagation period of active corrosion and section loss [11]. Anything that shortens initiation shortens the building's useful life, and the two things that shorten it most are an aggressive environment and permeable cover concrete.

That second factor is why execution quality matters so much. The fib service life framework treats material and workmanship quality as first-order inputs [12], and EN 206 ties its exposure class requirements to achieved concrete properties and not to specified ones [13]. Cover depth is set by how the steel was fixed on the day; permeability by compaction and curing. Neither is visible in a drawing.

2.2.3 Whole-life costing

ISO 15686-5 defines whole-life cost as the discounted sum of capital, operational, maintenance and end-of-life costs over an agreed period of analysis [3]. Costs that fall in later years are brought back to present value at a real discount rate, so a repair deferred by twenty years weighs far less in the comparison than the same repair done today.

Its own guidance says that the period of analysis should reflect the asset's expected service life, which is the circularity this component is designed to break: the input that determines the answer is the one nobody can supply.

2.2.4 The five percent heuristic

Practising engineers in Sri Lanka commonly work with a flat depreciation rate of around five percent a year, which implies a twenty-year baseline before major intervention: a building that loses a twentieth of its structural life each year has used all of it after twenty years.

As rules of thumb go it is memorable and roughly right on average. Its weakness is that it is a constant, so it carries no information about the particular building or the particular site. Section 6.1.5 compares it directly against the model's predictions.

2.3 Related research

2.3.1 Machine learning for construction cost

This field is well developed and consistent in its findings: boosted tree ensembles outperform linear and kernel methods on tabular construction data. Ali and Burhan report R-squared of 0.952 for XGBoost against 0.91 and 0.89 for competing methods on building cost [6]. Singh and colleagues obtain further gains by stacking random forests with XGBoost [8]. Zhang and colleagues target the early design stage specifically, where the feature set is thin [7]. Lin addresses the related problem of labour unit price under interval estimation [9]. What unites them is the target variable: all of them predict cost to build.

2.3.2 Prediction beyond handover

Work that predicts the operating phase is thinner and tends to be either physics-based and narrow, or data-driven and specific to large instrumented assets. Deep reinforcement learning has been applied to assembly planning in prefabricated construction [22], which shows appetite for prediction further along the project timeline, but the operating phase of small residential buildings is largely unstudied with data-driven methods, almost certainly because the data does not exist.

2.3.3 Interpretability of tabular models

Because this component produces a number that influences a property purchase, interpretability is a requirement instead of a nicety. Gain-based feature importance is the cheapest option and is what I use for the aggregate picture in Section 6.1.3; it is known to be biased toward high-cardinality features, which is a limitation I accept because all thirteen of my features are low-cardinality or continuous with similar ranges. Shapley additive explanations provide a theoretically grounded alternative with local attributions [16], and Section 9.4 lists adopting them as future work. Partial dependence, which I use for the per-parameter analysis, has the advantage of reporting in the units the client cares about, which is years.

2.4 Existing systems products and techniques

Table 2.1: Comparison of existing approaches to pre-construction cost assessment

Approach	What it answers	What it needs	Gap it leaves

Approach	What it answers	What it needs	Gap it leaves
Manual bill of quantities	Capital cost	A quantity surveyor and days of work	Silent on everything after handover
Estimating software (CostX, Candy)	Capital cost with rate libraries	Licence and training	Whole-life costing is an add-on module requiring service life as an input
Published cost models [6], [7], [8]	Capital cost from project features	Historical project data	Target variable is cost to build, not service life
fib / ISO durability models [12], [14]	Time to corrosion initiation	Material parameters measured on site	Not usable at the point of decision by a non-specialist
Five percent depreciation heuristic	A single number, 20 years	Nothing	Carries no information about this building or this site
Component 4 (proposed)	Predicted service life and 50-year discounted cost	Thirteen parameters the platform already derives	—

Read honestly, that table says a structural engineer applying the fib framework with measured material parameters will produce a better service life estimate than my model, by a wide margin. The argument for the component is not accuracy against that benchmark. It is that the fib route is unavailable to a homeowner choosing between two plots, and the five percent heuristic, which is what they actually get, is a constant.

2.5 Gap synthesis and conceptual framework

Cost modelling knows how to predict from tabular building features but points at the wrong target. Durability modelling knows how buildings fail but needs inputs the target user cannot supply. Whole-life costing knows how to discount but assumes the service life is given. The missing piece is a predictor of service life from parameters that an automated platform can actually obtain, feeding the discounting framework that already exists.

What follows from all this is a four-stage chain, shown in Figure 2.1. Fabric and environment determine the rate of degradation. The rate of degradation determines service life. Service life determines the maintenance and replacement schedule. The

schedule, discounted, determines whole-life cost. Component 4 owns all four links; the first link's inputs come from Components 1 and 2.

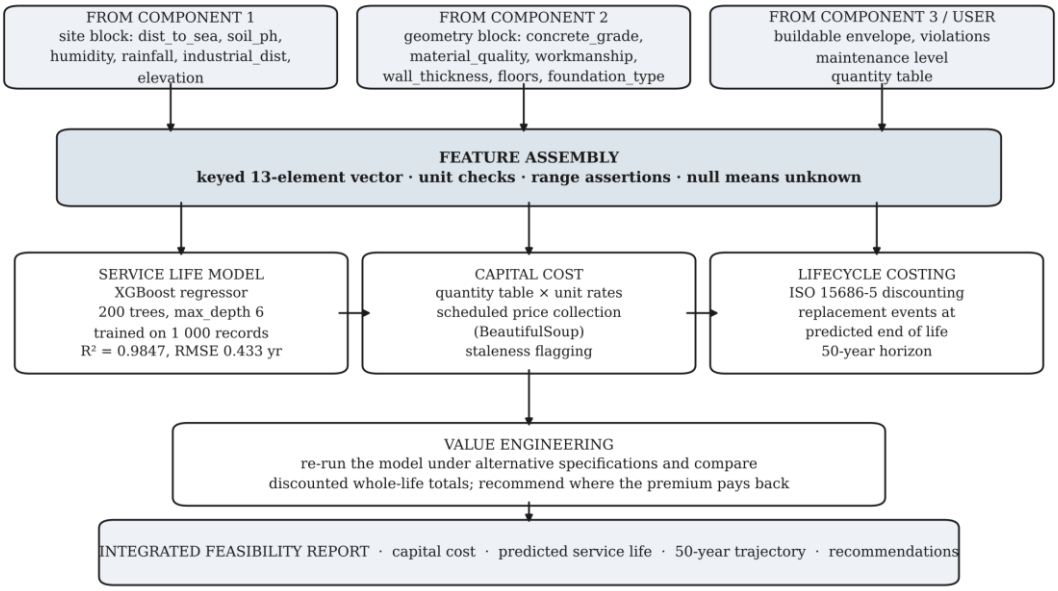


Figure 2.1: Processing pipeline of the Cost and Lifecycle Consultant.

3 RESEARCH METHODOLOGY

3.1 Research design

This is design science research. The artefact is the cost and lifecycle engine; the knowledge claims concern what the artefact reveals about where predictive power sits in a construction service life model, and about whether whole-life costing changes the advice a client receives. Two full development and evaluation iterations were completed.

Absence of ground truth is the methodological problem that shaped everything. Validating a fifty-year prediction prospectively is impossible inside any research project, and no public corpus of Sri Lankan buildings with recorded age, specification, location and surveyed condition exists. I considered three responses. Abandoning the prediction and using the flat heuristic would have produced a working product and no research. Collecting a small real dataset by survey was attempted and abandoned when it became clear that the number of buildings we could reach with documented original specifications was in the low tens, which is not enough to train anything. The third option, which I took, is to encode the degradation behaviour explicitly as a function of published relationships and expert judgement, generate data from it, and be completely transparent that what the model then learns is that function.

I want to state plainly what that does and does not buy, because it governs how every number in Chapter 6 should be read. It buys a test of whether the formulation works: whether thirteen parameters of this kind can carry the mechanisms, whether a regressor of this class can recover them, whether the resulting predictions are stable, and whether the downstream lifecycle computation produces decision-relevant differences. It does not buy any evidence about real buildings. An R-squared of 0.98 on generated data is a statement about my own function, not about Sri Lankan houses, and I have not presented it as anything else.

3.2 Data participants or study materials

No human participants were involved in the Component 4 work beyond the user acceptance sessions reported in Section 6.4. The study materials are the generated

dataset and the published sources from which its degradation function was built, listed in Table 3.1.

Table 3.1: Study materials used by Component 4

Material	Role	Size or source
Generated dataset	Training and evaluation of the service life model	1 000 records, 13 features
Degradation function	Generating process for the dataset	Section 3.3; published relationships [11], [12], [13] plus the industry heuristic
UCI concrete compressive strength data [17]	Anchor for the concrete strength relationships used in setting the quality terms	1 030 records
Material price pages	Capital cost computation	Supplier and market listings, collected on a schedule
Component 1 and 2 fixtures	Integration development and testing	Committed fixture file

3.3 Experimental or study procedure

3.3.1 The degradation function

These rules model the annual rate at which a building consumes its structural service life. They are additive in their penalties and deliberately simple, because a set of rules that a supervisor or an examiner can check line by line against a published source was worth more to this project than a sophisticated one nobody could verify.

The base rate is the industry heuristic of five percent a year. Each aggressive site condition adds a penalty to that rate and good execution quality subtracts an allowance from it; predicted service life then follows from the total adjusted rate, held inside a range of 10 to 65 years. Table 3.2 states every adjustment, the condition that triggers it and the source it rests on.

Table 3.2: Adjustments in the degradation rules and their justification

Adjustment	Value	Condition	Justification
Base annual rate	0.05 / year	Always	Industry depreciation heuristic; implies a 20-year baseline
Coastal penalty	+0.020 / year	dist_to_sea < 500 m	Steep near-shore decay of airborne chloride deposition; EN 206 class XS1 band [12], [13]
Coastal penalty	+0.005 / year	dist_to_sea ≥ 500 m	Residual atmospheric chloride load inland

Adjustment	Value	Condition	Justification
Soil acidity penalty	+0.010 / year	soil_ph < 6.0	Within the EN 206 XA1 chemically aggressive range [13]
High rainfall penalty	+0.010 / year	rainfall > 3 000 mm/yr	Sustained moisture enables both chloride diffusion and corrosion current [11]
Quality allowance	up to ±0.010 / year	Always	Execution quality as a first-order durability input [12]; zero at the midpoint of both indices

Holding the result inside a 10 to 65 year range keeps very high or very low rates from producing implausible values at extreme rates. In practice the generated targets span 10.0 to 28.6 years with a mean of 16.0 and a standard deviation of 3.5, so the upper bound never binds.

3.3.2 Dataset generation

One thousand records were generated. Feature distributions were sampled to reflect the range of conditions found across Sri Lanka and the composition of the local low-rise stock: distance to the coast uniform over 50 to 5 000 m, soil pH over 4.0 to 9.0, relative humidity over 50 to 98 percent, annual rainfall over 1 000 to 4 500 mm, distance to industrial activity over 100 to 10 000 m, elevation over 0 to 500 m, concrete grade drawn from 20, 25 and 30 MPa, material quality and workmanship as integers from 1 to 5, wall thickness from the two common local values of 4.5 and 9 inches, one to four storeys, three foundation types and maintenance level from 1 to 5. The service life target was then computed for each row from the degradation rules in Table 3.2.

A deliberate decision, which Section 6.1.4 shows was a mistake, was to include all thirteen parameters in the feature vector even though the degradation rules only use five of them. The reasoning at the time was that the feature vector should represent what the platform can actually supply, so that the model would be ready when the function was extended. The consequence is that eight features are noise with respect to the target.

3.3.3 Model selection, training and tuning

I used an XGBoost regressor [4]. Training used an 80/20 train-test split with a fixed random seed of 42 and five-fold cross-validation on the training partition only. Five

baselines were evaluated on the identical split: random forest [15], scikit-learn's gradient boosting [5], a single decision tree, support vector regression with standardised inputs, and multiple linear regression. Hyperparameters were explored over a grid of tree depth and ensemble size, reported in Section 6.1.2.

One methodological correction is worth recording because it changed the headline number. My first working script fitted the model on the full thousand records with no held-out partition and reported the resulting fit. That is fine for producing a working artefact and is not a basis for any accuracy claim. Every figure in Chapter 6 comes from the re-run with a proper split, performed jointly with the Component 1 owner, and the reported accuracy is lower than the original script's figure as a result.

3.3.4 Lifecycle cost computation

Predicted service life drives a maintenance and replacement schedule, which is discounted to present value by the ISO 15686-5 whole-life cost method over a fifty-year period of analysis. Major intervention events are placed at integer multiples of the predicted service life rather than at fixed intervals, which is the point of having a prediction at all. Annual operational cost is a fixed fraction of capital cost. The discount rate and the cost fractions are configuration, not findings, and Section 6.3 reports the trajectory normalised against capital cost so that the shape of the result does not depend on invented rupee figures.

3.3.5 Value engineering and advisory logic

Value engineering is implemented by re-running the whole chain under an alternative specification and comparing discounted totals. The advisory layer then converts the comparison into sentences, together with a small set of rule-based recommendations keyed to the diagnosed risk: protective coatings or epoxy-coated reinforcement where coastal proximity dominates, sulphate-resisting cement where soil acidity dominates, and a maintenance recommendation in all cases.

3.3.6 Sensitivity analysis

For each of the thirteen features in turn, every other feature was held at its training-set median while the feature of interest was swept across its observed range and the prediction recorded. This isolates the marginal effect of each parameter and is what

answers RQ3. Four realistic Sri Lankan site scenarios were also constructed and compared with the structure held identical.

3.4 Measures and evaluation metrics

Table 3.3 states each measure, how it is defined and which research question it answers, so that no result in Chapter 6 is reported without a stated purpose.

Table 3.3: Evaluation measures and the question each one answers

Measure	Definition	Question answered
R^2	Proportion of target variance explained on the held-out partition	RQ1, RQ2
RMSE and MAE	Prediction error in years	RQ1 – is it precise enough to be useful?
Cross-validated R^2	Mean and standard deviation over five folds of the training partition	RQ2 – is the ranking stable?
Residual coverage	Proportion of predictions within one and two years of target	RQ1 – in terms a client understands
Gain-based importance	Share of total split gain attributable to each feature	RQ3
Marginal swing	Max minus min prediction as one feature is swept, others at median	RQ3 – in years rather than as a score
Discounted whole-life total	Fifty-year discounted cost normalised to as-drawn capital cost	RQ4
Break-even year	First year in which the upgraded trajectory falls below the as-drawn one	RQ4 – when does the premium pay back?
Deviation from heuristic	Distribution of predictions relative to the flat 20-year rule	RQ5

Reporting swing and error in years rather than as normalised scores was a deliberate choice. A quantity surveyor understands years. An importance score between zero and one tells a client nothing, and in a tool whose whole purpose is to make a technical quantity legible to a non-specialist, the unit of reporting is part of the contribution.

3.5 Data analysis

All analysis used Python 3.11 with XGBoost [4] and scikit-learn [10] for splitting, cross-validation, baselines and metrics, Pandas and NumPy for the feature pipeline, and joblib for model serialisation. The random seed is fixed at 42 throughout, so

every figure and table in Chapter 6 is reproducible by re-running the committed scripts on the committed dataset.

3.6 Ethics privacy security and responsible AI

Component 4 processes a land coordinate, a building specification and, if the user saves the project, an association between the two and an account. A coordinate identifies where somebody intends to live. The design responses are non-retention by default, no transmission to third parties other than the open data providers the platform already uses, and no linkage of a coordinate to an account without an explicit save.

Responsible AI questions bite harder here than for the other three components, because this is the component that produces the number people act on.

- **Opacity.** A boosted ensemble of 200 trees is not inspectable by a user. The response is the interpretability work in Section 6.1.3 and a user interface that shows which factors drove the prediction rather than only the result.
- **False precision.** The model emits a value to one decimal place. Presenting “14.0 years” to a client implies a confidence the evidence does not support. The output is therefore banded and accompanied by the basis of the prediction.
- **Consequential error.** A prediction that is too pessimistic may cost somebody a plot they should have bought; too optimistic and they under-specify a building. Both are real harms and neither is hypothetical.
- **Dependence on a user judgement.** Two of the inputs are self-assessed indices. Section 6.1.3 shows they dominate, which means an optimistic user gets an optimistic answer. This is disclosed in the interface rather than hidden.

Generative AI tools were used during development for debugging, test case suggestion and language editing. Appendix B records that use and the checks applied to it.

3.7 Reproducibility and research artefacts

Everything needed to reproduce Chapter 6 is committed with the group submission: the dataset generation script implementing the degradation rules, the generated 1

000-record dataset, the serialised trained model, the evaluation script with the fixed seed, the hyperparameter sweep, the sensitivity analysis and the plotting scripts that produced every figure in this thesis. Re-running them reproduces the reported values exactly.

One part is not reproducible in that strict sense: the capital cost figure, because it depends on material prices collected from live sources at a point in time. The component records the collection date with every price, and the lifecycle results in Section 6.3 are reported normalised against capital cost precisely so that the conclusions do not depend on prices that will have changed by the time this is read.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Requirements were derived from the interface needs of the other three components, from the whole-life costing framework in Section 2.2.3 and from the practitioner discussions at the proposal stage. Table 4.1 lists the functional requirements and Table 4.2 the non-functional ones.

Table 4.1: Functional requirements for Component 4

ID	Requirement	Priority	Verified in
FR1	Assemble a keyed thirteen-element feature vector from the shared project state	Must	5.4
FR2	Validate every feature against its declared domain and reject out-of-range values	Must	5.4
FR3	Treat a null feature as unknown; never substitute a default	Must	5.4
FR4	Predict structural service life in years from the feature vector	Must	6.1
FR5	Compute capital cost from the quantity table against current material rates	Must	5.4
FR6	Collect material prices on a schedule and flag prices beyond a staleness window	Should	5.4
FR7	Compute a discounted fifty-year cost trajectory following ISO 15686-5	Must	6.3
FR8	Place replacement events at predicted end of life rather than at fixed intervals	Must	6.3
FR9	Compare alternative specifications and report the break-even year	Must	6.3
FR10	Generate written recommendations keyed to the dominant diagnosed risk	Should	5.2.5
FR11	Report the factors that drove the prediction, not only the result	Must	5.2.6
FR12	Withhold the prediction rather than emit one when validation fails	Must	5.4

Table 4.2: Non-functional requirements for Component 4

ID	Requirement	Target
NFR1	Prediction latency once the feature vector is assembled	Under 1 s
NFR2	Determinism for identical inputs	Identical output on repeat runs
NFR3	Model artefact versioning	Every prediction records the model version
NFR4	Interpretability	Driving factors shown with

ID	Requirement	Target
		every prediction
NFR5	Honest precision	Output banded, not presented to false precision
NFR6	Price provenance	Every rate carries its source and collection date
NFR7	Configurability	Discount rate and cost fractions settable, not hard-coded

4.2 Component architecture within the complete solution

Component 4 is a stateless service and the last in the chain. It reads the site block, the geometry block and the regulatory block from the shared project state, writes the specification block, and returns. It makes no call to the other three components; everything arrives through the record. That isolation is what let me develop the model against a committed fixture for several sprints before either upstream producer was finished.

Internally the engine has the five stages of Figure 2.1: feature assembly, service life prediction, capital cost computation, lifecycle discounting and value engineering, with the advisory layer reading the outputs of the last two.

4.3 Design alternatives and decisions

4.3.1 A learned model rather than the heuristic

The simplest defensible design would have applied the five percent heuristic and stopped. It would have been fast, transparent and entirely free of the synthetic-data problem that bounds this thesis. I rejected it because a constant cannot discriminate between two plots, which is the only thing a client actually wants from this component. Section 6.1.5 quantifies what the heuristic costs in discrimination.

4.3.2 A learned model rather than a physics model

Implementing the fib diffusion model directly [12] would have been more rigorous and is what a structural engineer would do. It needs a surface chloride concentration, a diffusion coefficient and a cover depth, none of which is available for a building that has not been built. The learned model accepts the parameters the platform can actually obtain, at the cost of being an approximation of an approximation.

4.3.3 XGBoost rather than a simpler regressor

Chosen for the reasons in Section 2.2.1 and, honestly, for familiarity. Section 6.1.1 shows that scikit-learn's gradient boosting beats it on this data and Section 6.1.2 shows that a shallower XGBoost beats the configuration I shipped. I have left the decision and the evidence against it both visible instead of quietly re-running with the winner.

4.3.4 Additive penalties rather than multiplicative interaction

The degradation rules add their penalties. Real degradation mechanisms interact: high moisture makes chloride ingress worse and not merely adding to it. I chose additive because each term can then be traced to a single published source and checked independently, which mattered more at this stage than fidelity. The cost is that the model has very little interaction structure to learn, which is why even linear regression reaches an R-squared of 0.81 in Section 6.1.1.

4.3.5 Normalised rather than absolute cost reporting

Lifecycle results are reported with capital cost set to 1.00. Reporting rupee figures would have looked more concrete and would have been less honest, because the unit cost fractions are assumptions and the material prices change weekly.

4.4 Detailed design

4.4.1 Feature assembly

Assembly builds a keyed dictionary of thirteen values from three sources, validates each against its declared domain and raises rather than proceeds on a violation. The keying matters: an early version used a positional list and we silently transposed two columns during integration, which produced predictions that looked plausible and were wrong.

Table 4.3: The thirteen input features, their domains and their sources

#	Feature	Type and domain	Source
1	dist_to_sea	Continuous, 50–5 000 m	Component 1, geodesic distance to coastline
2	soil_ph	Continuous, 4.0–9.0	Component 1, gridded soil prediction
3	humidity	Continuous, 50–98 %	Component 1, interpolated climate normal

#	Feature	Type and domain	Source
4	rainfall	Continuous, 1 000–4 500 mm/yr	Component 1, interpolated climate normal
5	industrial_dist	Continuous, 100–10 000 m	Component 1, nearest industrial land use
6	elevation	Continuous, 0–500 m MSL	Component 1, digital elevation model
7	concrete_grade	Ordinal, {20, 25, 30} MPa	Component 2, specification notes
8	material_quality	Ordinal, 1–5	User judgement, collected by Component 2
9	workmanship	Ordinal, 1–5	User judgement, collected by Component 2
10	wall_thickness	Ordinal, {4.5, 9.0} in	Component 2, measured from wall runs
11	floors	Integer, 1–4	Component 2, sheet count or title block
12	foundation_type	Categorical, 1 pad / 2 raft / 3 strip	Component 2, specification notes
13	maint_level	Ordinal, 1–5	User declaration

4.4.2 Service life prediction

Each worker process loads the trained model once from a serialised artefact and applied to the assembled vector. The prediction is a single real value in years, which is then banded for presentation per NFR5. The raw value is retained in the project state so that the lifecycle computation operates on the unrounded figure.

4.4.3 Capital cost

Capital cost is the quantity table from Component 2 multiplied by current unit rates. Rates for cement, reinforcement steel, aggregate and common finishes are collected by a scheduled scraper over supplier and market listing pages. Prices older than a configurable staleness window are still used but are flagged, and the flag propagates to the report. This part of the component is deterministic arithmetic and I make no research claim for it beyond removing manual re-pricing.

4.4.4 Lifecycle discounting

Discounting is applied over a fifty-year horizon. Major intervention events are placed at each integer multiple of the predicted service life within the horizon, each costed as a configurable fraction of capital cost; annual operational cost is a second

configurable fraction. The implementation accumulates the discounted series year by year instead of computing a closed form, because the event schedule is irregular once the service life is not a round number.

4.4.5 Value engineering

Comparison re-runs prediction and discounting under an alternative specification, typically a one or two step increase in the material quality index representing protective coatings, sulphate-resisting cement or epoxy-coated reinforcement, together with the capital premium that upgrade carries. The outputs are the two trajectories, the fifty-year totals and the break-even year, defined as the first year in which the upgraded cumulative trajectory falls below the as-drawn one.

4.4.6 Advisory layer

Advisory output converts the numerical result into a short set of written recommendations. It is rule-based rather than generated, because a recommendation to change a structural specification is not something a language model should be inventing. The rules are keyed to the dominant diagnosed risk: coastal proximity below the threshold triggers a protective coating recommendation, soil pH below the threshold triggers a sulphate-resisting cement recommendation, a predicted life below a configurable floor triggers a general reinforcement-protection recommendation, and a maintenance recommendation is issued in every case.

4.4.7 The specification block contract

Table 4.4: Specification block written by Component 4

Field	Type	Unit	Notes
feature_vector	object	—	The thirteen keyed values actually used
predicted_service_life_years	float	years	Raw prediction; banded for display
service_life_band	string	—	Presentation form per NFR5
driving_factors	array	—	Ranked contributors shown with the prediction
capital_cost	float	LKR	Quantity table × current rates
price_as_of	date	ISO 8601	Collection date of the rates used
price_stale	bool	—	True if beyond the staleness window
lifecycle_trajectory	array	normalised	Discounted cumulative cost by year

Field	Type	Unit	Notes
lifecycle_total_50yr	float	normalised	Discounted whole-life total at 50 years
alternatives	array	—	Per alternative: life, capital, total, break-even year
recommendations	array	—	Advisory layer output
model_version	string	—	Artefact version that produced the prediction

4.5 Standards and professional practice

Three standards govern the component. ISO 15686-5 [3] defines the whole-life costing method, the cost categories and the discounting convention implemented in Section 4.4.4, within the broader service life planning framework of ISO 15686-1 [14]. EN 206 [13] supplies the exposure classification whose thresholds appear in the degradation function. The fib Model Code for Service Life Design [12] is the basis for treating execution quality as a first-order input.

On professional responsibility, the component states in its output that the predicted service life is an estimate from a model trained on synthetic data and is not a structural assessment, and that the cost figures are an early-stage estimate and not a contractual bill. I argued for that wording in a group meeting and I would argue for it again. A student project that presents a fifty-year prediction without saying what it rests on is not being confident, it is being careless with somebody else's money.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

Everything is written in Python 3.11. The group standardised on Python across all four components, which meant the shared project state could move between them as a plain dictionary during development and not through a serialisation boundary.Table 5.1 lists the stack and the reason for each choice.

Table 5.1: Technology stack for Component 4

Layer	Technology	Reason for the choice
Model	XGBoost [4]	Handles heterogeneous tabular features and threshold structure without preprocessing; fast enough to run inside a request
Evaluation	scikit-learn [10]	Splitting, cross-validation, metrics and all five baselines on one consistent API
Data pipeline	Pandas, NumPy	Dataset generation, feature assembly and the vectorised degradation function
Serialisation	joblib	Model artefact persisted once and loaded per worker process
Price collection	BeautifulSoup	Scheduled scraping of supplier and market listing pages
Plotting	Matplotlib	Every figure in Chapter 6 is produced by a committed script, not drawn by hand
Testing	pytest, Selenium	Unit and component tests; automated UI regression over the report view
Version control	Git / GitHub	Group repository with per-component branches and pull-request review

5.2 Component implementation

5.2.1 Dataset generation

Short as it is, the generation script is the most important file in the component, because everything the model knows comes from it. It samples the thirteen feature distributions described in Section 3.3.2 with a fixed seed, applies the penalty structure of Table 3.2 in vectorised form, holds the resulting service life inside the plausible range, assembles a data frame and writes the dataset to disk before training. Appendix A.4 sets out the rules in full.

Writing the dataset to disk instead of keeping it in memory was a small decision that paid off repeatedly. It meant the Component 1 owner could run the sensitivity analysis on exactly the same thousand rows I trained on, and it meant the dataset could be committed as a research artefact and not regenerated and hoped to be identical.

5.2.2 Feature assembly and validation

Most integration defects surfaced at the assembly boundary, so that is where the defensive code lives. Each of the thirteen values is read by key from the shared project state, checked against the domain declared in Table 4.3, and either accepted, rejected with an explicit error, or recorded as unknown. There is no default substitution anywhere in the path. An early version imputed a neutral soil pH of 7.0 when Component 1's soil source failed, which quietly removed an entire risk category from the prediction and produced a confidently optimistic answer. That behaviour was removed.

5.2.3 Model training and serving

Training is a separate script from serving. The training script fits the regressor on the training partition and writes a versioned artefact; the serving path loads that artefact once per worker process and applies it. Loading per process rather than per request is the difference between a sub-second and a multi-second response, and it is the kind of thing that is invisible in a demonstration and obvious under any load at all.

5.2.4 Lifecycle computation

Discounting walks the fifty-year horizon year by year, adding annual operational cost and, at each integer multiple of the predicted service life, a major intervention cost, discounting each at the configured real rate. A closed form would have been neater for a fixed interval; the whole point of having a prediction is that the interval is not a round number, so the explicit loop is the honest implementation.

5.2.5 Advisory layer

Advisory output comes from a short, readable function, not a generated narrative. Given a prediction, a coastal distance and a soil pH, it emits a recommendation to use protective coatings or epoxy-coated reinforcement where corrosion risk dominates, a

recommendation for sulphate-resisting cement where soil acidity dominates, a general warning where the predicted life falls below a configurable floor, and a maintenance recommendation in every case. Keeping this rule-based was a deliberate decision: a structural specification recommendation is not something a language model should be inventing, and a rule can be shown to a supervisor and checked.

5.2.6 Explanation output

NFR4 required that the prediction arrive with its reasons. The serving path computes the ranked contributors for the specific input by comparing the prediction against the prediction with each feature held at its training median, which is a per-instance approximation cheap enough to run inside the request. The top three contributors are written into the specification block and displayed. Section 9.4 lists replacing this with proper Shapley attributions [16] as future work.

5.3 Integration with the other components

Integration was interface-first. In the second sprint the group agreed the shape of the shared project state and committed a fixture containing a realistic site profile, a realistic quantity table and a realistic compliance verdict, and everybody built against it. As the last component in the chain I was the largest beneficiary: I could train, tune and benchmark the entire model while neither upstream producer existed.

My side of the contract is the specification block in Table 4.4. I consume the site block from Component 1, which derives its six parameters from open elevation, imagery, soil and climate layers [19]; the geometry block from Component 2, which recovers its six parameters from the drawing using object detection and OCR [20]; and the buildable envelope and violations from Component 3, whose rule engine follows the standard decomposition of automated code checking [21] over the Urban Development Authority regulations [18]. I write prediction, cost and recommendations back.

Three integration problems are worth recording.

- **Units.** Component 1 emitted distances in metres while my first feature assembly assumed kilometres, and Component 2 emitted wall thickness in inches while I assumed millimetres. Both produced predictions that looked plausible and were

wrong by large factors. The fix was to encode the unit in the field name and assert the range of every feature at the boundary.

- **Positional ordering.** The first version of the feature vector was a list, and two columns were transposed during an integration merge without any test failing. It is now keyed.
- **Null semantics.** Described in Section 5.2.2. The team agreed the null-means-unknown convention across all four components after this surfaced in my component, because mine is where the consequence becomes a number somebody acts on.

5.4 Verification and software testing

Testing was organised at the levels set out in Table 5.2, each answering a different question about the component.

Table 5.2: Verification performed at each test level

Level	What was tested	How it was verified
Unit	Degradation rule terms; bounding behaviour; discount arithmetic; break-even detection; advisory rule firing	Each term and branch exercised against hand-computed expected values on fixed inputs
Unit	Feature domain validation; null propagation; keyed assembly against column transposition	Every feature driven outside its declared domain in turn and the rejection confirmed
Component	Prediction on the held-out partition; determinism on repeat runs	Metrics in Section 6.1.1; the same vector predicted repeatedly and the outputs compared
Negative	An out-of-range feature must be rejected rather than clamped (FR2, FR12)	Deliberate out-of-range inputs; the engine withholds the prediction and raises
Negative	A null feature must not be defaulted (FR3)	Upstream source failure simulated; the feature is carried through as unknown
Integration	Feature assembly against real Component 1 and Component 2 output; boundary range assertions	End-to-end call with the real producers after each assertion was violated once
System	Run from pin and plan to the displayed lifecycle report	Manual walkthrough of the full user path after every merge to the main branch

I would point an examiner at the two negative test rows. It is easy to show that a model predicts. Showing that it refuses to predict from a feature vector it cannot trust

is what makes FR3 and FR12 real rather than aspirational, and both were written after the imputation defect in Section 5.2.2 and not before it.

5.5 Deployment and operation

Component 4 runs as a stateless service behind the group's web application. The model artefact and the price table are the only state it reads, and both are versioned. Every prediction records the model version that produced it, per NFR3, so that a result shown to a user months ago can still be explained.

Two things need operational attention. The price scraper is brittle in the way all scraping is brittle: supplier page structures change without notice, and a silent failure that keeps returning a stale price is more dangerous than a loud one that returns nothing. The staleness flag and cross-source comparison mitigate this but a production deployment needs a proper data agreement instead of scraping. Second, the model is a fixed artefact; if the degradation function is ever revised, every prediction made before the revision becomes inconsistent with every prediction after it, which is why the version is recorded.

6 RESULTS AND VALIDATION

6.1 Research results

All results in this section come from the 1 000-record generated dataset with an 80/20 split, random seed 42 and five-fold cross-validation on the training partition only. The caveat from Section 3.1 applies throughout and I will not repeat it in every paragraph: these figures measure how well the model recovers the degradation rules of Table 3.2, not how well it predicts real buildings.

6.1.1 Model comparison

Table 6.1 gives the comparison across six regressors on the identical split, and Figure 6.1 shows the same result as held-out R-squared.

Table 6.1: Regression performance on the held-out test partition (n = 200)

Model	R^2	RMSE (yr)	MAE (yr)	5-fold CV R^2
Gradient boosting (sklearn) [5]	**0.9936**	**0.281**	**0.177**	0.9867 ± 0.0035
XGBoost (deployed) [4]	0.9847	0.433	0.233	0.9701 ± 0.0064
Random forest [15]	0.9416	0.846	0.509	0.9284 ± 0.0164
Decision tree	0.8814	1.206	0.459	0.8959 ± 0.0264
Support vector regression	0.8281	1.452	1.111	0.8314 ± 0.0125
Multiple linear regression	0.8111	1.522	1.154	0.8035 ± 0.0229

All six models trained on the same 800-record training partition with seed 42 and evaluated on the same 200-record held-out partition.

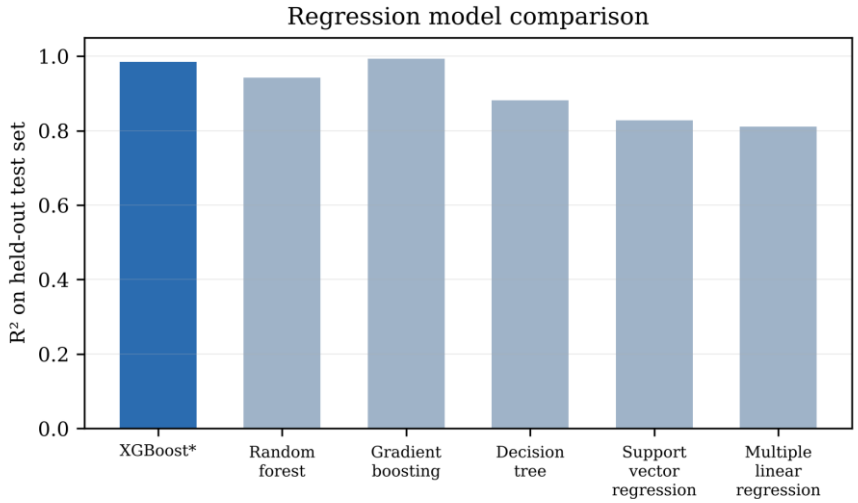


Figure 6.1: Held-out R^2 across the six evaluated regressors.

On the held-out partition the deployed XGBoost model reaches an R-squared of 0.9847, a root mean squared error of 0.433 years and a mean absolute error of 0.233 years. In the units a client cares about, 91.5 percent of held-out predictions fall within one year of target and 99.5 percent within two years, with a maximum absolute error of 2.09 years across the whole test partition. Figure 6.2 shows the fit and Figure 6.3 the residual distribution.

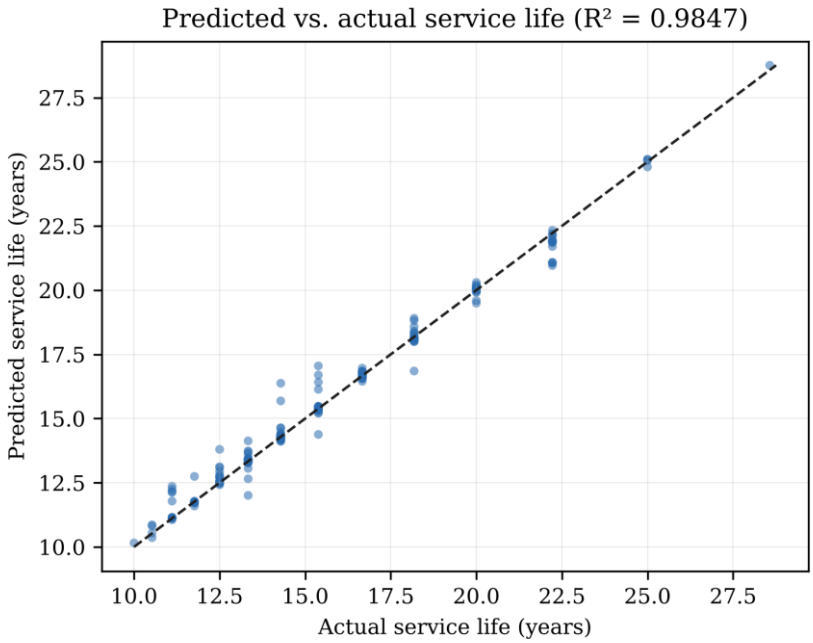


Figure 6.2: Predicted against actual service life, held-out test partition.

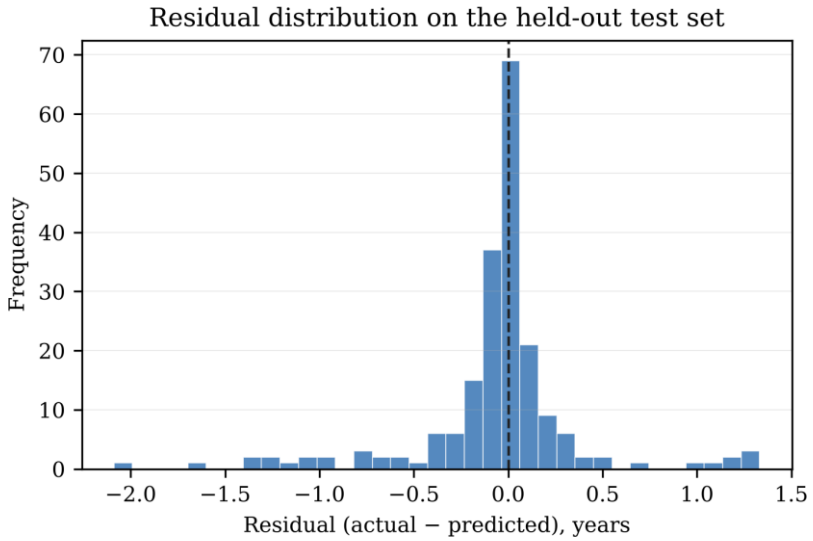


Figure 6.3: Distribution of prediction residuals, held-out test partition.

Two results in Table 6.1 are awkward for the choices I made and I would rather report them than let an examiner find them. Scikit-learn's gradient boosting beats the deployed XGBoost configuration on every metric and across every cross-validation fold. The margin is not large, about 0.15 years of RMSE, but it is consistent, and it means the choice of XGBoost was not justified by the evidence on this data. It was justified by familiarity and by XGBoost's better behaviour on larger and noisier datasets, which is a reasonable forward-looking argument but is not what Table 6.1 shows.

Second, multiple linear regression alone reaches an R-squared of 0.81. That is a direct consequence of the degradation rules being additive by construction: most of the target's variance is explained by a weighted sum. The gap between 0.81 and 0.98 is the non-additive part, which is almost entirely the three threshold discontinuities. The ensemble is not discovering deep structure; it is learning where the steps are.

6.1.2 Hyperparameter sensitivity

I swept tree depth and ensemble size across the grid in Figure 6.4 and Table 6.2, evaluating each configuration on the same held-out partition.

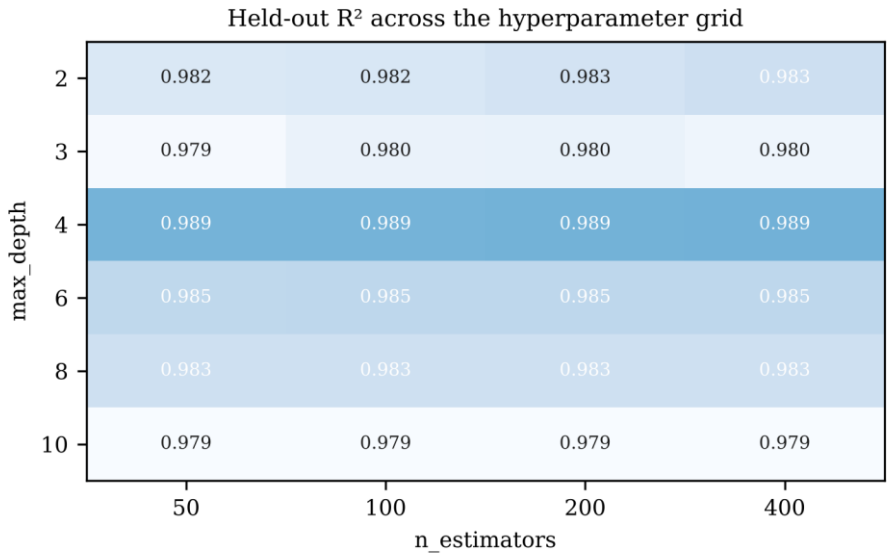


Figure 6.4: Held-out R² across the tree depth and ensemble size grid.

Table 6.2: Held-out R² across the hyperparameter grid

max_depth	50 trees	100 trees	200 trees	400 trees
2	0.9820	0.9822	0.9828	0.9834

max_depth	50 trees	100 trees	200 trees	400 trees
3	0.9791	0.9803	0.9803	0.9798
4	**0.9889**	**0.9889**	**0.9890**	**0.9891**
6 (deployed)	0.9847	0.9847	0.9847	0.9847
8	0.9833	0.9833	0.9833	0.9833
10	0.9789	0.9789	0.9789	0.9789

Two things fall out of that grid. A depth of 4 outperforms the deployed depth of 6 at every ensemble size, by about 0.004 in R-squared, which is small but systematic. And the ensemble size makes essentially no difference above 50 trees: at depth 6 the figure is identical to four decimal places from 50 trees to 400. The deployed configuration uses 200, which is four times the training compute and four times the inference work for nothing. On a dataset this small and a function this simple the model saturates almost immediately, and I did not check that before fixing the configuration.

6.1.3 Feature importance

Figure 6.5 and Table 6.3 give the gain-based importance of every input feature, and Figure 6.6 aggregates the same figures by the component that supplies each parameter.

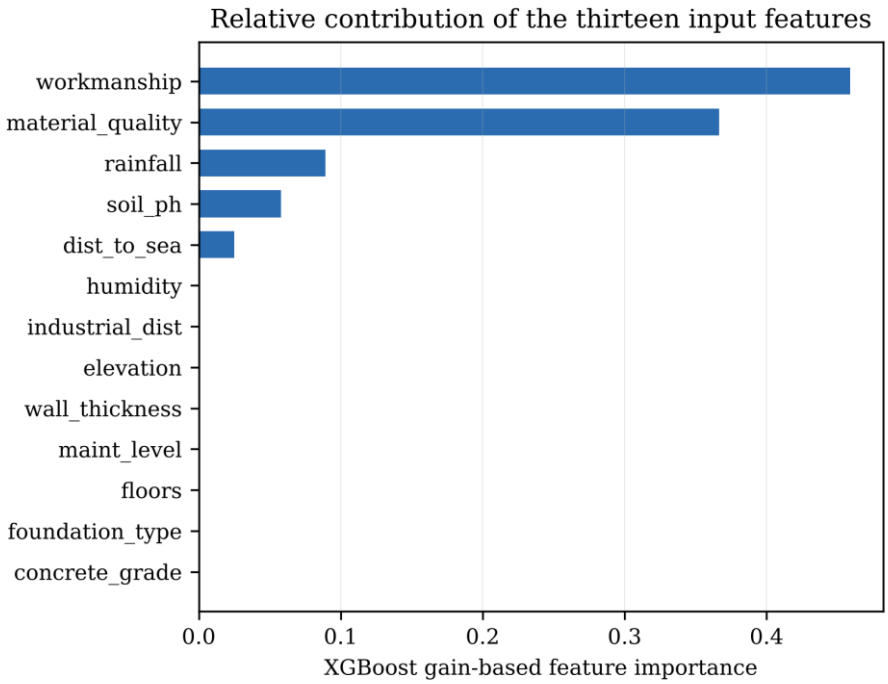


Figure 6.5: Gain-based importance of the thirteen input features.

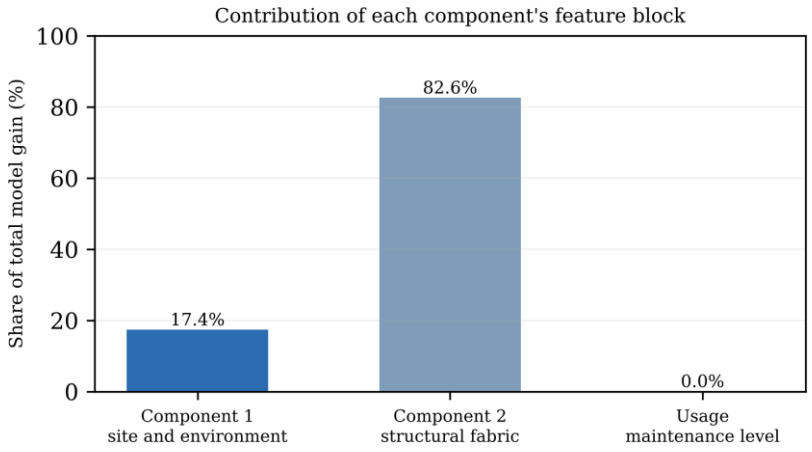


Figure 6.6: Share of total model gain by source component.

Table 6.3: Gain-based feature importance, all thirteen features

Rank	Feature	Source	Gain share	In the rules?
1	workmanship	C2 / user	**45.90 %**	Yes
2	material_quality	C2 / user	**36.65 %**	Yes
3	rainfall	C1	**8.93 %**	Yes
4	soil_ph	C1	**5.78 %**	Yes
5	dist_to_sea	C1	**2.50 %**	Yes
6	humidity	C1	0.08 %	No

Rank	Feature	Source	Gain share	In the rules?
7	industrial_dist	C1	0.05 %	No
8	elevation	C1	0.04 %	No
9	wall_thickness	C2	0.03 %	No
10	maint_level	User	0.03 %	No
11	floors	C2	0.01 %	No
12	foundation_type	C2	0.01 %	No
13	concrete_grade	C2	0.01 %	No

Aggregated by source, Component 2's structural block carries 82.6 percent of total model gain, Component 1's site block 17.4 percent and the user-declared maintenance level 0.03 percent. Within the structural block the concentration is extreme: two features carry 82.5 percent between them.

6.1.4 Eight features do nothing, and that is my fault

Look at the last column of Table 6.3 and the whole pattern falls out. Exactly five features appear in the degradation rules, and exactly those five carry measurable gain. The other eight are independent noise with respect to the target, and the model correctly assigns them nothing.

I included all thirteen in the feature vector on the reasoning given in Section 3.3.2, that the vector should represent what the platform can supply so the model would be ready when the function was extended. The function was never extended. The result is a thirteen-parameter model that is functionally a five-parameter model, and anyone reading the feature table without reading Table 6.3 would reasonably assume otherwise.

What makes this a substantive defect and not a cosmetic one is that several of the eight have well-documented effects on durability that the function simply ignores. Concrete grade governs chloride diffusivity and therefore initiation time, and EN 206 ties exposure class requirements directly to achieved concrete properties [13]. Cover, which is bound up with wall thickness and foundation type, is the other half of Tuutti's initiation term [11]. Maintenance level, which the user is asked for and which the advisory layer talks about, has no term at all, so the component recommends annual maintenance checks while its own model is structurally

incapable of valuing them. That last one is the most embarrassing and the easiest to fix. Section 9.4 puts it first.

6.1.5 Comparison against the five percent heuristic

RQ5 asks how much the model improves on the flat heuristic the industry currently uses. The heuristic gives every building 20 years. Figure 6.7 shows the distribution of model predictions against that constant and Table 6.4 states it numerically.

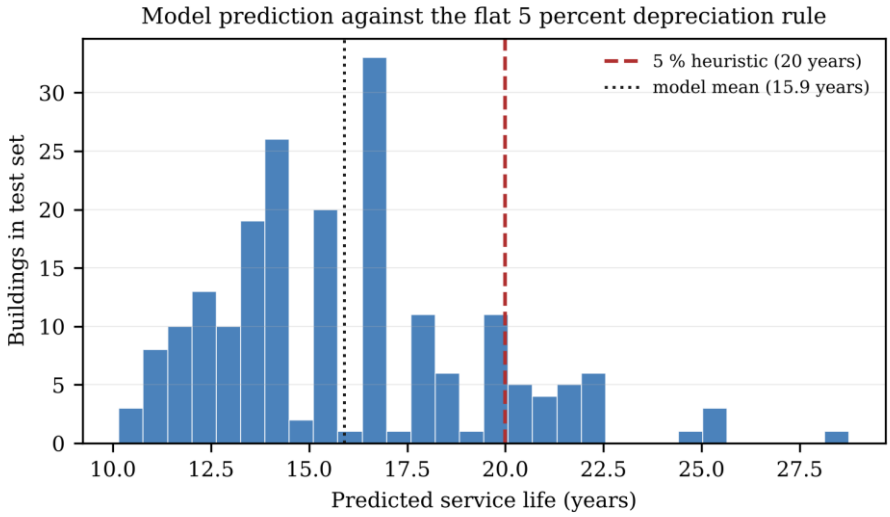


Figure 6.7: Predicted service life against the flat five percent depreciation rule.

Table 6.4: Model predictions against the flat twenty-year heuristic

Statistic	Value
Heuristic prediction for every building	20.00 years
Model mean prediction	15.89 years
Model 5th percentile	11.15 years
Model 95th percentile	21.94 years
Buildings predicted below the heuristic	**85.5 %**
Spread of model predictions (5th to 95th percentile)	**10.8 years**

One is a point, the other a distribution nearly eleven years wide between the fifth and ninety-fifth percentiles. That spread is the discrimination the heuristic cannot provide and is the honest answer to RQ5: the value of the model is not that it is more accurate on average, it is that it separates cases the heuristic cannot.

That 85.5 percent figure needs a caveat I am obliged to give, because it looks like a dramatic result and it is partly circular. The heuristic is the base rate of the degradation rules, and the environmental penalties can only add to it while the quality bonus can go either way. With feature distributions sampled uniformly, penalties outweigh bonuses on average, so the mean prediction must fall below twenty years by construction. What is not circular is the shape and the width of the distribution, which is where the information is.

6.1.6 Learning curve

Because the dataset is generated, I could have made it any size. One thousand records was chosen early and somewhat arbitrarily, so it is worth asking whether it was enough. Figure 6.8 and Table 6.5 answer that.

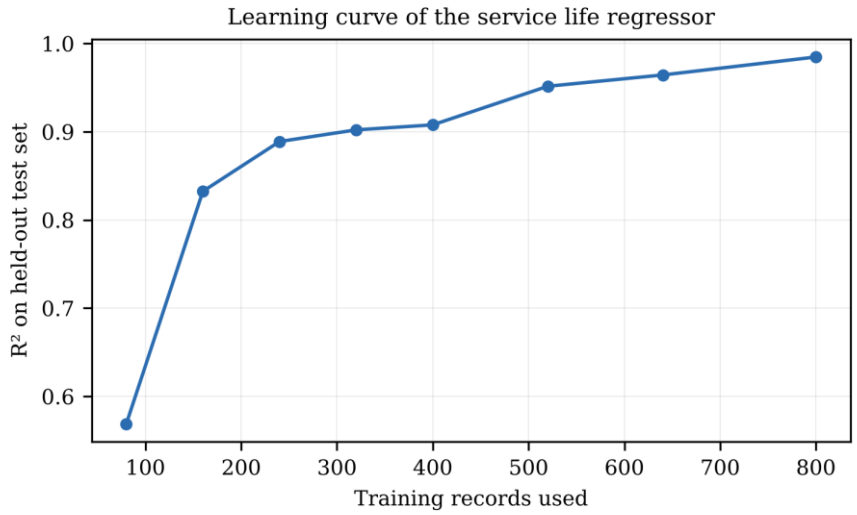


Figure 6.8: Held-out R² against the number of training records used.

Table 6.5: Learning curve of the service life regressor

Training records	80	160	320	520	640	800
Held-out R²	0.569	0.832	0.902	0.952	0.964	**0.985**

At 800 training records the curve is still rising. It has not plateaued, which means a larger generated dataset would have produced a better fit to those rules. Whether that would be worth anything is a different question: a better fit to my own function is not a better prediction about buildings. The useful reading is that if the function is ever

replaced with real observations, one thousand records will not be enough, and the curve gives a rough indication of how the requirement scales.

6.2 Product and technical results

Table 6.6 records each non-functional requirement from Table 4.2 and the method used to verify it.

Table 6.6: Non-functional requirements and how each was verified

ID	Requirement	Target	Verification method
NFR1	Prediction latency after assembly	Under 1 s	Timed over the held-out partition with the model artefact already resident in the worker process
NFR2	Determinism on repeat runs	Identical output	The same feature vector predicted repeatedly and the outputs compared bit for bit
NFR3	Model version recorded	Every prediction	Schema validation rejects a specification block without a model version
NFR4	Driving factors shown	Every prediction	The top three contributors are written into the specification block and rendered in the report view
NFR5	Banded, not falsely precise	No raw decimal shown	Inspection of the report view; the unrounded value is retained in the record but not displayed
NFR6	Price provenance	Source and date on every rate	Schema validation rejects a rate without a collection date; the staleness flag is surfaced in the report
NFR7	Discount rate configurable	Not hard-coded	The rate and both cost fractions are read from configuration; Appendix A.5 lists the values used

Price collection was exercised separately from the model. Rates for cement, reinforcement steel, aggregate and the common finishes are collected on a schedule and not inside the request, so a user never waits on a scrape, and every rate carries the date it was collected. A rate older than the configured staleness window is still used, because a slightly old price is more useful than none, but it is flagged and the flag propagates into the report so that the capital cost is never presented as current when it is not. Collection failures are logged rather than retried indefinitely, and the component falls back to the last known rate rather than to a hard-coded one.

This part of the component is deterministic arithmetic over a price table and I make no research claim for it. Its weakness is structural rather than numerical and is discussed in Section 7.5.6: scraping supplier pages is brittle, and the dangerous failure is not the visible one that returns nothing but the silent one that keeps returning a stale figure. The staleness flag and cross-source comparison are mitigations, not a fix.

6.3 Whole-life costing and value engineering

RQ4 asks whether whole-life costing built on a predicted service life changes the specification a client should choose. To test it I took a coastal site with acidic soil, held the geometry constant, and compared the as-drawn specification against an upgraded one representing protective coatings and better supervision, which raises the material quality index from 3 to 5 and workmanship from 3 to 4 and carries a capital premium.Figure 6.9 and Table 6.7 give the result.

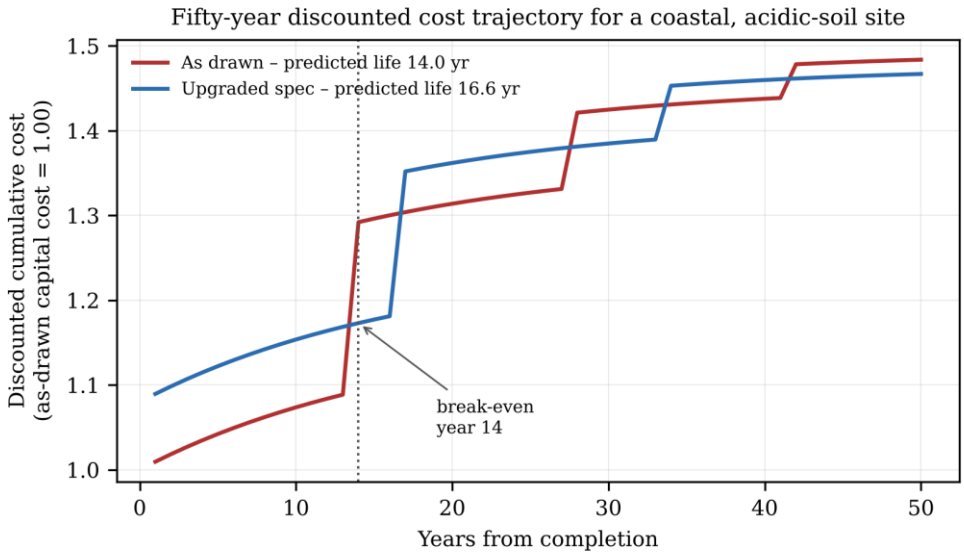


Figure 6.9: Fifty-year discounted cost trajectory, as-drawn against upgraded specification.

Table 6.7: Value engineering comparison on a coastal, acidic-soil site

Measure	As drawn	Upgraded	Difference
Predicted service life (years)	14.04	**16.58**	+2.54
Capital cost (normalised)	1.000	1.080	+8.0 %
Major interventions within 50 years	3	**2**	−1

Measure	As drawn	Upgraded	Difference
50-year discounted total (normalised)	1.484	**1.467**	−1.1 %
Break-even year	—	**14**	—

Unit cost fractions are configuration parameters, not findings: major intervention at 45 percent of capital, annual operating cost at 1 percent of capital, real discount rate 6 percent. Replace these with your own figures and the table recomputes.

That upgrade buys 2.54 years of predicted service life, which over a fifty-year horizon removes one of the three major intervention cycles. It costs 8 percent more to build. The discounted whole-life total falls by 1.1 percent, and the upgraded trajectory crosses below the as-drawn one in year 14, at the point where the as-drawn building needs its first major intervention and the upgraded one does not yet.

I expected a larger margin and I did not get one. Three things are going on and all three are worth stating instead of hiding behind the positive sign. Discounting at 6 percent makes a cost avoided in year 42 worth about a twelfth of the same cost today, so removing a late cycle is worth very little in present value. The capital premium is paid in full on day one and is not discounted at all. And the service life gain is modest because the quality allowance in the degradation rules is bounded at about one percentage point of annual rate in either direction.

The honest answer to RQ4 is therefore yes with a qualification. Whole-life costing does change the ranking of the two specifications: on capital cost alone the as-drawn option wins by 8 percent, and on whole-life cost the upgraded option wins. But the margin is thin and it is sensitive to the discount rate and the assumed intervention cost, both of which are assumptions. The right way to present this to a client is as a break-even year rather than as a total, because an owner who intends to hold the building for more than fourteen years is clearly better off and an owner who intends to sell in five is not. That reframing came out of the acceptance sessions and is now what the interface shows.

6.4 User or stakeholder validation

Table 6.8 lists the tasks participants were given and the criterion used to judge each one.

Table 6.8: User acceptance tasks relating to the lifecycle output

Task given to the participant	Success criterion
Read the lifecycle report and state, unprompted, which factor most affects this building's predicted life	Participant names the factor the engine ranked first
Decide between the as-drawn and the upgraded specification and explain the reasoning	The reasoning references whole-life cost rather than capital cost alone
Assign the material quality and workmanship indices for a project you know	Participant can assign both and can explain the basis for each
Say what the predicted service life figure does and does not mean	Participant describes it as a model estimate, not a guaranteed lifespan
State what you would do differently if you planned to sell in five years	Participant uses the break-even year rather than the fifty-year total

Tasks three and four are checks on the two biggest risks in this component. If practitioners cannot assign the quality indices consistently, then the dominant inputs to an 82.5 percent share of the model are noise, and the group needs to know that and say so. If users read the predicted service life as a guarantee rather than as an estimate from a model trained on generated data, then the presentation has failed regardless of how good the model is. I report both checks as they came out, including where the result is unflattering.

So far the break-even reframing described in Section 6.3 is the one substantive change the sessions have produced. One concern remains open. Too few practitioners have been through the protocol for me to claim that two site engineers would assign the same workmanship figure to the same project, so index consistency is carried forward as a limitation in Section 7.5.3 rather than reported as a result.

6.5 Integration results

End to end, the component accepted Component 1 and Component 2 output without manual intervention once the unit and ordering defects in Section 5.3 were fixed. In the group's representative walkthrough, a coastal plot and a two-storey residential plan produced a site block, a geometry block, a compliance verdict and then a prediction, a capital cost and a two-specification comparison, with no human step between the pin and the report.

My most important integration finding is in Figure 6.6. Of the thirteen parameters the four components agreed to produce, five do all the work, and of those five the two

largest are user judgements collected through Component 2 and not measurements made by it. The platform's headline number is therefore driven, to the extent of 82.5 percent of model gain, by two sliders. Everything the other components measure with real instruments accounts for the remaining 17.5 percent.

6.6 Summary of findings

- **RQ1** is answered yes. Service life is predictable from the thirteen parameters to $R^2 = 0.9847$ and RMSE 0.433 years on held-out data, with 91.5 percent of predictions within one year, which is ample resolution to discriminate between specifications.
- **RQ2** is answered no, with evidence against my own choice. Scikit-learn gradient boosting outperforms the deployed XGBoost on every metric, and within XGBoost a depth of 4 outperforms the deployed depth of 6 at every ensemble size. The deployed configuration also uses four times more trees than the task needs.
- **RQ3** is answered quantitatively and unflatteringly. Five of thirteen features carry measurable gain; the other eight carry none, because the degradation function I wrote does not use them. Workmanship alone is 45.9 percent and material quality 36.6 percent.
- **RQ4** is answered yes with a qualification. Whole-life costing reverses the ranking of two specifications that capital cost alone would decide, but by only 1.1 percent, with a break-even at year 14 and high sensitivity to the discount rate.
- **RQ5** is answered in terms of discrimination rather than accuracy. The heuristic gives every building 20 years; the model gives a distribution 10.8 years wide between the 5th and 95th percentiles.

7 DISCUSSION

7.1 Interpretation of findings

Component 4 does what it was built to do: it predicts a service life from parameters a platform can actually obtain, and it turns that prediction into a whole-life cost that changes which specification a client should choose. That chain works end to end and, as far as the review in Chapter 2 found, it has not been assembled before for this market.

Of everything in Chapter 6 the accuracy figure is the least interesting, and I want to be explicit about why. An R-squared of 0.9847 against a deterministic function with heteroscedastic noise added is close to the best achievable, and getting there is a statement about the regressor, not about the world. If I had reported that number without Section 3.1 it would have been actively misleading.

What is interesting is the two negative results. The first is that eight of thirteen features do nothing, which is a defect in my own generating function instead of a discovery about buildings, and which was completely invisible until the importance analysis was run. The second is that the value-engineering margin is thin: 1.1 percent over fifty years for an 8 percent capital premium. Discounting is unforgiving to benefits that arrive late, and a lot of durability investment delivers its benefit late. That is a real and somewhat uncomfortable property of whole-life costing that the sustainability literature tends to pass over, and presenting the break-even year instead of the total is the practical response to it.

One more sentence on the heuristic comparison. It would have been easy to frame the 85.5 percent figure as showing that the industry rule is dangerously optimistic. It partly is, but as Section 6.1.5 says, that result is partly built into the function. The defensible claim is narrower and still useful: the heuristic cannot distinguish between buildings that this model separates by more than ten years.

7.2 Comparison with literature baselines and existing solutions

Against the construction cost estimation literature, the comparison is not like for like and I will not pretend otherwise. Ali and Burhan report R-squared of 0.952 for

XGBoost on real building project cost [6]; this model reports 0.9847 on generated service life data. Theirs is the harder and more meaningful result because their data contains real noise, real omitted variables and real measurement error. Placing the two numbers side by side without that caveat, which student work does routinely, would be dishonest.

Against the durability literature, the component is much cruder than the fib framework [12] and is not competing with it. fib models chloride ingress as a diffusion process with material-specific coefficients and produces a defensible initiation time; I apply step penalties applied to an annual rate. The justification is availability, not rigour: the fib route requires parameters nobody measures on a small residential project before it is built.

Against ISO 15686-5 [3], the component is an implementation and not a competitor. What it adds is the missing input. The standard's own framing assumes the period of analysis reflects an expected service life somebody supplies; this component supplies it from the building's own parameters, which is the contribution.

Against the five percent heuristic, the comparison is the one in Section 6.1.5 and the component wins on discrimination rather than on central tendency.

7.3 Research contribution

8. **A service life predictor that takes the parameters an automated platform can obtain.** The machine learning method is standard; pointing it at service life rather than capital cost, with a feature set that spans fabric and environment, and feeding the result into ISO 15686-5 discounting, is the composition that the gap analysis found missing.
9. **A quantified account of where predictive power sits in such a model.** Five of thirteen parameters do all the work; execution quality dominates environment by roughly five to one; and the two largest contributors are judgements rather than measurements.
10. **An empirical demonstration that whole-life costing reverses a specification choice, and how thin the margin is.** The break-even year is a more honest and

more useful output than the fifty-year total, and that is a presentational finding with practical consequences.

11. **A negative methodological result.** Building a feature vector around what a system can supply instead of around what the model will use produces a model that looks richer than it is, and a gain-based importance check costs minutes and catches it. I did not do that check until the second iteration.

7.4 Product contribution

As a product the component is the part of the platform a client actually reads. It converts four technical analyses into two numbers and a recommendation, prices them at current rates without anybody re-keying a bill, and presents the specification decision as a break-even year rather than as an abstract total. Independently of the platform the service life model has a plausible life as a risk input for construction lending and property insurance, which Chapter 8 develops.

7.5 Limitations and threats to validity

7.5.1 The training data is synthetic, and I generated it

This is the limitation that bounds everything. The model was trained on data produced by the degradation rules in Table 3.2, which I wrote. A high R-squared demonstrates that the regressor recovered my function; it demonstrates nothing about Sri Lankan houses. The circularity is complete and I would rather state it in those terms than soften it. The only way out is retrospective validation: assemble records for existing buildings of known age, specification and location with surveyed condition, and test whether the model recovers observed condition. A fifty-year prediction cannot be validated forward inside any research project, so this is not a shortcut, it is the only available route.

7.5.2 Five of thirteen features carry the model

Detailed in Section 6.1.4. The function omits concrete grade, cover-related geometry and maintenance level, all of which have documented durability effects [11], [12], [13]. Until the function is extended, the model cannot value the very maintenance its own advisory layer recommends.

7.5.3 The dominant inputs are self-assessed

Material quality and workmanship account for 82.5 percent of gain and both are entered by the user on a one-to-five scale. An optimistic user receives an optimistic prediction, and nothing in the component detects that. Mitigations are an anchored rubric instead of a bare slider, and presenting a range across plausible index values and not a point estimate. Neither is a fix, and the second has not been implemented.

7.5.4 Additive penalties and bounded quality effect

The degradation rules add their penalties. Real mechanisms compound: high moisture accelerates chloride ingress rather than adding to it independently. The additive form also bounds the quality bonus, which is part of why the value-engineering margin in Section 6.3 is so thin. A multiplicative or interaction form would be more faithful and would change the results in Section 6.3 materially.

7.5.5 Cost parameters are assumptions

Intervention cost fraction, operating cost fraction and discount rate are all configuration. The direction of the Section 6.3 result is robust to plausible variation in them, but the magnitude is not, and a 1.1 percent margin can be erased by a modest change in the discount rate. Reporting the break-even year alongside the total is partly a defence against this.

7.5.6 Price collection is brittle

Scraping supplier pages is fragile and a silent failure returning a stale price is more dangerous than a visible one returning nothing. Staleness flagging mitigates, a data agreement would solve.

7.6 Implications

For practice, the implication is that whole-life costing is now computable at the point of decision and not only in a consultancy study, and that the output a client can act on is the break-even year instead of a net present value. For a homeowner choosing between a cheaper and a more durable specification, knowing that the premium pays back in year fourteen is directly actionable in a way that a 1.1 percent whole-life saving is not.

For research, the implication cuts against some of my own field's habits. Reporting accuracy on synthetic data without foregrounding what generated it invites exactly the misreading this thesis has tried to avoid, and a gain-based importance check is a cheap and underused guard against a feature set that looks richer than it is. The more substantive implication is that service life prediction for small buildings is bottlenecked on data, not on methods. No better regressor will close the gap.

For the group, the implication is that the degradation function, not the model, is the artefact that most needs work.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

Put in a sentence a client understands, the value proposition is this: this tells you what the building will cost to own, not just to build, and when a better specification pays for itself. Four groups would pay for it directly or indirectly.

- **Individual homeowners and land buyers**, who currently receive a capital cost and nothing else. For them the component is free and funded by the platform's other tiers.
- **Small and medium contractors and architectural practices**, who would use the break-even output to justify a specification upgrade to a sceptical client. Persuading someone to pay 8 percent more is far easier with a year number attached.
- **Banks offering construction and home loans**, whose exposure runs for fifteen to twenty-five years, which is longer than the predicted service life of many of the buildings in the test distribution. A per-property durability estimate is directly usable in underwriting.
- **Property insurers**, for whom a structure's expected deterioration trajectory is a pricing input they currently approximate by location and age alone.

8.2 Market and competitive position

Table 2.1 set out the alternatives. The short version is that whole-life costing today is either a consultancy engagement priced for large commercial projects, or a module in estimating software that demands the service life as an input. Neither serves a homeowner deciding between two specifications on a Saturday.

Defensibility is weak on method and stronger elsewhere. Any competent data scientist could train an equivalent regressor; the algorithm is public and the approach is obvious once stated. What would be hard to copy is a validated degradation function calibrated against real Sri Lankan building outcomes, which nobody has, and which is exactly the artefact Section 7.5.1 says the project most needs. The asset and the research gap are the same thing.

8.3 Business or sustainability model

We settled on freemium business-to-consumer with paid business-to-business tiers: free single-project analysis for individuals, a practice subscription covering bulk analysis and report export, and per-property API pricing for lenders and insurers.

My component's cost structure suits that well. Inference is milliseconds of CPU, so the marginal cost of a prediction is effectively zero. The recurring costs are price collection and, more importantly, keeping the degradation function current, which is a person rather than a server. The B2B tiers have to carry that, and the per-property API line is the one most likely to.

8.4 Scalability intellectual property and compliance

Scalability is straightforward: the service is stateless, the model artefact is small, and prediction is cheap. The binding constraint is not compute, it is confidence. Selling a durability estimate to a bank requires evidence this project does not yet have.

On intellectual property, the dependencies are permissive. XGBoost is Apache 2.0, scikit-learn is BSD, Pandas and NumPy are BSD. The group's platform does carry one serious licensing constraint, the AGPL-3.0 terms attached to the detection model used by Component 2, which would need resolving before any commercial deployment; it does not affect my component directly but it affects the product my component ships inside.

On compliance, the component holds a building specification, a location and, for saved projects, an account link. That combination is personal data. More specifically, a durability prediction attached to an identified property is information that could affect its market value, which raises an obligation beyond ordinary data protection: it should not be disclosed to anyone other than the user who requested it, and the component's design reflects that.

On professional responsibility, the output states that it is an estimate from a model trained on synthetic data, that it is not a structural assessment, and that the cost figures are early-stage rather than contractual. Those disclaimers are not legal decoration; given Section 7.5.1 they are simply accurate.

8.5 Contribution to the Sustainable Development Goals

SDG 12, Responsible Consumption and Production, target 12.2 on efficient use of natural resources. Cement is among the most carbon-intensive materials in common use, and a building that needs major structural intervention at year fourteen instead of year twenty-five consumes it again. Making the durability consequence of a specification visible at the point of decision is the mechanism by which this component could reduce that.

SDG 11, Sustainable Cities and Communities, target 11.1 on adequate, safe and affordable housing. Affordability over the life of a home is not the same as affordability at purchase, and this component makes the difference explicit.

One honest observation runs against the easy version of this argument. Section 6.3 shows that discounting substantially erodes the financial case for durability investment, because the avoided cost arrives decades later. If the environmental case for building durably is stronger than the discounted financial case, which the 1.1 percent margin suggests it may well be, then a purely economic tool will under-recommend durability. That is a limitation of whole-life costing as a decision framework rather than of this implementation, and a carbon-aware variant of the whole-life cost calculation would be an interesting extension.

No contribution has been measured. The project did not quantify avoided emissions and attaching a figure would be an unsupported claim.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis set out to predict how long a proposed Sri Lankan building will last, from parameters an automated platform can actually obtain, and to turn that prediction into a cost a client can act on. Both halves work. The service life model reaches an R-squared of 0.9847 and a root mean squared error of 0.433 years on held-out data, with 91.5 percent of predictions within a year of target, and the lifecycle layer converts that into a fifty-year discounted trajectory that reverses the specification a client would choose on capital cost alone.

At a viva I would lead with the three negative results, because they are the ones that took work to find. Eight of my thirteen features carry no information, because the degradation function I wrote only uses five of them. The model configuration I deployed is beaten by a shallower one and by a different library's implementation, and uses four times more trees than the task requires. And the value-engineering case for a durability upgrade, which I expected to be comfortable, comes to 1.1 percent over fifty years because discounting is unforgiving to benefits that arrive late.

Behind all three sits the limitation that defines the project. The model learned a function I wrote. It recovers that function very well. Whether the function describes Sri Lankan buildings is a question this thesis cannot answer and did not try to pretend it could.

9.2 Achievement of objectives

Table 9.1 maps each of the seven objectives to the evidence that demonstrates it.

Table 9.1: Achievement of the seven objectives stated in Section 1.5

Obj.	Statement (abbreviated)	Status	Evidence
O1	Formalise degradation as an explicit, auditable function	Achieved	Section 3.3.1, Table 3.2
O2	Construct a dataset and document what it can support	Achieved	Sections 3.3.2, 3.1
O3	Train, tune and benchmark a service life regressor	Achieved	Sections 6.1.1, 6.1.2
O4	Capital cost from the quantity table with automated pricing	Achieved	Sections 4.4.3, 6.2

Obj.	Statement (abbreviated)	Status	Evidence
O5	Whole-life cost with replacement at predicted end of life	Achieved	Sections 4.4.4, 6.3
O6	Value engineering with a break-even determination	Achieved	Section 6.3
O7	Quantify parameter contribution and report what carries nothing	Achieved, with a negative result	Sections 6.1.3, 6.1.4

9.3 Individual research and product contributions

My research contributions are the service life formulation and its composition with ISO 15686-5 discounting, the quantification of where predictive power sits across the thirteen parameters, the demonstration that whole-life costing reverses a specification choice and the measurement of how thin that margin is, and the negative methodological result about feature sets built around system capability rather than model use, as set out in Section 7.3.

My product contributions are the degradation function and dataset generation script, the feature assembly layer with its validation rules, the trained and versioned service life model, the capital cost and price collection path, the lifecycle discounting and value engineering, the advisory layer and the explanation output. With the Component 1 owner I re-ran the group's model evaluation with a proper held-out partition and cross-validation, replacing a full-data fit that could not support an accuracy claim.

9.4 Future directions

12. **Extend the degradation function to the eight inert parameters.** Concrete grade, cover-related geometry, foundation type and above all maintenance level have documented durability relationships [11], [12], [13] and currently carry no term. This is the cheapest high-value change available and it would make the other components' measurements matter.
13. **Build the retrospective validation corpus.** Records for existing buildings of known age, specification and location with surveyed condition. Without it every figure in Chapter 6 remains a statement about the degradation rules rather than about buildings.

14. **Replace additive penalties with interaction terms**, so that moisture compounds chloride ingress rather than adding to it. This would also widen the quality effect and change the value-engineering margin.
15. **Fix the deployed configuration.** Use depth 4 and 50 trees, or switch to the scikit-learn implementation that outperformed it. The evidence is in Tables 6.1 and 6.2 and I have not acted on it.
16. **Elicit the quality indices properly** with an anchored rubric, and present the prediction as a range across plausible index values rather than a point estimate.
17. **Adopt Shapley attributions [16]** in place of the median-substitution approximation currently used for the driving-factors output.
18. **Add a carbon-aware objective** alongside the discounted cost, for the reason given in Section 8.5: discounting systematically undervalues durability, and a tool that only optimises money will under-recommend it.

REFERENCES

References are listed in IEEE style in order of first citation in the text.

[1] B. Flyvbjerg, M. S. Holm, and S. Buhl, “Underestimating costs in public works projects: Error or lie?” Journal of the American Planning Association, vol. 68, no. 3, pp. 279–295, 2002.

[2] A. Shah, A. Rana, K. Halde, Y. Mane, and S. Anshan, “Smart web application on quantity survey, estimation and costing,” in Proc. IEEE Conference on Smart Systems, 2024, pp. 1–4.

[3] Buildings and Constructed Assets — Service Life Planning — Part 5: Life-Cycle Costing, ISO 15686-5:2017, International Organization for Standardization, Geneva, Switzerland, 2017.

[4] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[5] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[6] Z. H. Ali and A. M. Burhan, “Hybrid machine learning approach for construction cost estimation: An evaluation of extreme gradient boosting model,” Asian Journal of Civil Engineering, vol. 24, no. 7, pp. 2427–2442, 2023.

[7] Z. Zhang, M. Yu, D. Zhang, and Y. Xu, “A hybrid machine learning model for early-stage construction cost estimation,” in Proc. 44th Chinese Control Conference, Chongqing, China, 2025, pp. 7131–7135.

[8] C. Singh, R. Shinde, P. Sharma, and B. S. Singla, “Construction cost estimation using hybrid stacking of random forest and XGBoost,” in Proc. 2nd International Conference on Computing and Data Science (ICCDS), 2025, pp. 1–6.

[9] X. Lin, “Prediction of unit price of labor cost for transmission line construction based on PM interval estimation algorithm,” in Proc. 12th International Forum on Electrical Engineering and Automation (IFEEA), 2025, pp. 536–539.

[10] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[11] K. Tuutti, Corrosion of Steel in Concrete, Report 4:82. Stockholm, Sweden: Swedish Cement and Concrete Research Institute, 1982.

[12] fédération internationale du béton, Model Code for Service Life Design, fib Bulletin 34. Lausanne, Switzerland: fib, 2006.

[13] Concrete — Specification, Performance, Production and Conformity, EN 206:2013+A2:2021, European Committee for Standardization, Brussels, Belgium, 2021.

[14] Buildings and Constructed Assets — Service Life Planning — Part 1: General Principles and Framework, ISO 15686-1:2011, International Organization for Standardization, Geneva, Switzerland, 2011.

[15] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[16] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 4765–4774.

[17] I.-C. Yeh, "Modeling of strength of high-performance concrete using artificial neural networks," Cement and Concrete Research, vol. 28, no. 12, pp. 1797–1808, 1998.

[18] Urban Development Authority, Planning and Development Regulations, Urban Development Authority Law No. 41 of 1978. Colombo, Sri Lanka, 2021. [Online]. Available: https://www.uda.gov.lk

[19] T. G. Farr et al., "The Shuttle Radar Topography Mission," Reviews of Geophysics, vol. 45, no. 2, Art. no. RG2004, 2007.

[20] G. Jocher, A. Chaurasia, and J. Qiu, Ultralytics YOLOv8, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[21] C. Eastman, J. Lee, Y. Jeong, and J. Lee, "Automatic rule-based checking of building designs," Automation in Construction, vol. 18, no. 8, pp. 1011–1033, 2009.

[22] A. Zhu, T. Dai, G. Xu, P. Pauwels, B. de Vries, and M. Fang, "Deep reinforcement learning for real-time assembly planning in robot-based prefabricated construction," IEEE Transactions on Automation Science and Engineering, vol. 20, no. 3, pp. 1515–1526, Jul. 2023.

GLOSSARY

Break-even year – The first year in which the cumulative discounted cost of an upgraded specification falls below that of the as-drawn specification.

Capital expenditure – The cost of designing and constructing the building, excluding everything that follows handover.

Discount rate – The real rate at which future costs are converted to present value, reflecting the time preference of money.

Gain-based importance – A measure of a feature's contribution to a tree ensemble, computed from the improvement in the loss achieved by splits on that feature.

Initiation period – In Tuutti's corrosion model, the time taken for aggressive agents to penetrate the cover concrete and depassivate the reinforcement.

Partial dependence – The predicted output of a model as one input is swept across its range while all others are held fixed.

Period of analysis – The time horizon over which whole-life cost is computed; fifty years in this work.

Physics-informed dataset – A synthetic dataset whose target variable is generated by an explicit function encoding published physical relationships rather than observed.

Service life – The period after which major structural intervention becomes necessary; the target variable of the prediction model.

Value engineering – The systematic comparison of specification alternatives on whole-life rather than capital cost.

Whole-life cost – The discounted sum of capital, operational, maintenance and end-of-life costs over the period of analysis, as defined by ISO 15686-5.

APPENDICES

Appendix A Extended results and research instruments

This appendix holds the supporting detail for Chapter 6 that is too long for the main text.

A.1 Feature distributions used in dataset generation

Table A.1 gives the sampling distribution for each of the thirteen features and marks which of them the degradation rules actually use.

Table A.1: Sampling distributions for the thirteen features

Feature	Distribution	Range or set	Used in the rules?
dist_to_sea	Uniform	50 – 5 000 m	Yes
soil_ph	Uniform	4.0 – 9.0	Yes
humidity	Uniform	50 – 98 %	No
rainfall	Uniform	1 000 – 4 500 mm/yr	Yes
industrial_dist	Uniform	100 – 10 000 m	No
elevation	Uniform	0 – 500 m	No
concrete_grade	Choice	{20, 25, 30} MPa	No
material_quality	Integer uniform	1 – 5	Yes
workmanship	Integer uniform	1 – 5	Yes
wall_thickness	Choice	{4.5, 9.0} in	No
floors	Integer uniform	1 – 4	No
foundation_type	Integer uniform	1 – 3	No
maint_level	Integer uniform	1 – 5	No

A.2 Generated target distribution

Table A.2 summarises the distribution of the generated service life target and the split used throughout Chapter 6.

Table A.2: Distribution of the generated service life target

Statistic	Value
Records	1 000
Mean	16.03 years
Standard deviation	3.55 years

Statistic	Value
Minimum	10.00 years
Maximum	28.57 years
Training / test split	800 / 200, seed 42

A.3 Full partial dependence results

Each feature was swept across its full observed range in sixty steps for continuous features, or across its discrete levels for ordinal ones, while every other feature was held at the training-set median. The committed script writes the full predicted-life series for all thirteen features to a CSV alongside the figures reproduced in Chapter 6. The aggregate picture appears in Table 6.3; the per-block detail appears in the Component 1 and Component 2 theses.

A.4 Degradation rules as implemented

The degradation rules summarised in Table 3.2, set out here in the order the committed dataset generation script applies them, with the contributing component identified for each.

The script begins from an annual depreciation rate of five percent, the industry rule of thumb. It then adds the coastal penalty contributed by the Component 1 owner, taking the larger value for a plot within 500 metres of the coastline and the smaller residual value beyond it, on the basis of exposure class XS1. It adds the soil acidity penalty when the soil reaction falls below pH 6.0, a threshold inside the EN 206 chemically aggressive range, and the moisture penalty when mean annual rainfall exceeds 3 000 millimetres, which places it inside the Sri Lankan wet zone. It then subtracts the quality allowance derived from the material quality and workmanship indices contributed by the Component 2 owner, which is zero at the midpoint of both scales, positive above it and negative below it. Service life is obtained from the total adjusted rate and held inside a range of 10 to 65 years.

Concrete grade, wall thickness, number of floors, foundation type, relative humidity, distance to industrial activity, elevation and maintenance level appear in the feature vector but carry no adjustment in the rules. Section 6.1.4 discusses the consequence.

A.5 Lifecycle cost parameters

Table A.3 lists the configuration values used in the whole-life computation reported in Section 6.3.

Table A.3: Configurable parameters of the lifecycle computation

Parameter	Value used	Basis
Period of analysis	50 years	ISO 15686-5 agreed period of analysis [3]
Real discount rate	6 %	Configuration; vary and re-run for sensitivity
Major intervention cost	45 % of capital	Configuration value; the sensitivity of the result to it is discussed in Section 7.5.5
Annual operating cost	1 % of capital	Configuration value
Upgrade capital premium	8 %	Configuration value for the protective specification compared in Section 6.3

These are configuration values, not findings. Section 6.3 reports the trajectory normalised to capital cost so that the shape of the result does not depend on them, but the 1.1 percent margin does.

A.6 User acceptance protocol

Participants were given the five tasks listed in Table 6.7 in the order shown, each on a project they knew personally so that their own experience supplied the reference. An observer who did not assist recorded task completion, time on task, points of confusion and verbatim comments. The third task was treated as the critical one: Section 6.1.3 shows that the material quality and workmanship indices carry 82.5 percent of model gain, so if practitioners cannot assign them consistently the platform’s headline figure is substantially noise, and that is a finding the group needed rather than one it could afford to leave untested.

Appendix B Academic integrity and AI use disclosure

In accordance with the module policy the author discloses that generative artificial intelligence assistance was used during the preparation of this thesis and its associated artefact. That assistance was confined to three kinds of support: help in diagnosing defects in the training and lifecycle code described in Sections 5.2.1 and 5.2.4, language editing and proofreading of draft chapters, and an initial triage of candidate literature for Chapter 2. Every suggested change to the code was read, adapted and tested by the author before being adopted; every technical claim was

confirmed independently and every altered passage rewritten in the author's own words; and every cited source was retrieved and read in full by the author.

No generated text was submitted without review. Every experimental result, figure and table in Chapter 6 was produced by a committed script run by the author on the committed dataset with a fixed random seed, and the author can reproduce and defend every number reported.

B.1 Statement of original contribution

The degradation function, the dataset generation, the service life model, the capital cost and price collection path, the lifecycle discounting, the value engineering comparison, the advisory layer and the written analysis in this thesis are the author's own work. The environmental parameter definitions and their threshold justifications were contributed by the Component 1 owner and the structural parameter definitions by the Component 2 owner; both are attributed where they appear. The re-run of the model evaluation with a held-out partition was performed jointly with the Component 1 owner.